Documento de Arquitectura de Software (DAS) de SITRA: Servicio de Integración y Trazabilidad para una Cooperativa de Ahorro y Crédito

Jorge Mario Vélez Vásquez

Programa de Ingeniería de Sistemas, Corporación Universitaria Remington

Arquitectura de Software

Gloria Amparo Lora Patiño

7 de noviembre de 2026

Repositorio del proyecto: https://github.com/JorgeMario-V/sitra

Resumen

Una cooperativa de ahorro y crédito debe conectar su sistema central de cuentas, construido hace 20 años y sin documentación, con una billetera digital externa, y demostrar ante el regulador que todas sus operaciones quedan registradas sin posibilidad de alteración. El cliente impuso tres condiciones: no modificar ese sistema, no invertir dinero adicional y no contar con personal técnico. Este documento describe la arquitectura de SITRA (Servicio de Integración y Trazabilidad), la solución propuesta: una aplicación organizada en capas que se comunica con el sistema central por medio de un conector intercambiable, protege cada petición con un token firmado y guarda una bitácora de auditoría que la propia base de datos escribe y protege con huellas encadenadas. Se presentan los requisitos y atributos de calidad, la estrategia y la tecnología elegidas, cinco vistas de arquitectura, cinco decisiones de arquitectura documentadas, el corte vertical ejecutable con sus pruebas (30 pruebas automáticas de la API y 21 pruebas unitarias y de integración) y la evaluación de compromisos y riesgos. La solución cumple los escenarios de seguridad, fiabilidad y mantenibilidad usando únicamente software libre y equipos que la cooperativa ya tiene.

Palabras clave: arquitectura de software, sistema legado, trazabilidad, auditoría, API REST, seguridad, cooperativa

Tabla de contenido

Documento de Arquitectura de Software (DAS) de SITRA: Servicio de Integración y Trazabilidad para una Cooperativa de Ahorro y Crédito....5

Control de versiones del documento....7

Glosario de términos técnicos....8

Sección 1. Introducción y drivers arquitectónicos....12

1.1 Visión general del negocio y alcance....13

1.2 Historias de usuario arquitectónicamente significativas....15

1.3 Atributos de calidad prioritarios....16

1.4 Restricciones....18

Sección 2. Estrategia arquitectónica y tecnológica....20

2.1 Patrón arquitectónico principal: N-Capas....20

2.2 Pila tecnológica....23

Sección 3. Vistas de arquitectura (modelo C4 y UML)....25

3.1 Vista de contexto (C4, nivel 1)....26

3.2 Vista de contenedores y capas (C4, nivel 2)....27

3.3 Vista de dominio: diagrama de clases UML....29

3.4 Vista dinámica: diagrama de secuencia UML de la HU-01....32

3.5 Vista de despliegue....34

Sección 4. Registro de decisiones de arquitectura (ADR)....37

ADR-001: Patrón N-Capas con un conector intercambiable para el núcleo legado....38

ADR-002: MariaDB (la del XAMPP existente) en lugar de PostgreSQL....40

ADR-003: La trazabilidad la garantiza la base de datos, no solo el código....41

ADR-004: En el CRUD de Transferencia, actualizar (PUT) es “corregir y reintentar”.....43

ADR-005: Token JWT firmado con HMAC del JDK y secretos con BCrypt, sin Spring Security completo.....45

Sección 5. Entregable de programación ejecutable (corte vertical).....47

5.1 Repositorio Git.....48

5.2 Script SQL de la base de datos (migraciones en MariaDB).....49

5.3 Código del backend por capas (Spring Boot y JPA).....53

5.4 Pruebas de la arquitectura.....58

Sección 6. Evaluación de la arquitectura y compromisos.....62

6.1 Análisis de compromisos.....62

6.2 Puntos de sensibilidad y riesgos técnicos.....64

6.3 Conclusión arquitectónica.....65

Conclusiones.....66

Referencias.....68

Anexo A. Lista de chequeo de entrega.....70

Anexo B. Correcciones de la versión 1.1.....71

Documento de Arquitectura de Software (DAS) de SITRA: Servicio de Integración y Trazabilidad para una Cooperativa de Ahorro y Crédito

Una cooperativa de ahorro y crédito opera sobre un **núcleo transaccional legado**: el programa central que lleva las cuentas de los asociados y mueve su dinero, construido hace 20 años en un lenguaje de programación propietario, sin documentación y sin nadie en el equipo actual que lo conozca por dentro. El ente regulador le exige dos cosas en un plazo de seis meses: conectar ese núcleo con una **billetera digital externa** (una aplicación de pagos de otra empresa que usan los asociados) y entregar **trazabilidad**, es decir, poder demostrar quién hizo cada operación, sobre qué cuenta, cuándo y con qué resultado, con evidencia de que nadie la alteró después.

Este documento describe la **arquitectura de software** de SITRA (Servicio de Integración y Trazabilidad), la solución propuesta para ese problema. La arquitectura de un sistema es su organización de alto nivel: qué piezas lo forman, qué responsabilidad tiene cada una, cómo se comunican y, sobre todo, por qué se eligió esa organización y no otra. El documento también registra lo que se sacrificó en cada decisión y los riesgos que quedan abiertos.

El texto está pensado para dos tipos de lectores: el equipo técnico que mantenga el sistema y personas de la cooperativa o del regulador sin formación en programación. Por eso cada sigla o término técnico se explica en palabras sencillas la primera vez que aparece, y todos se reúnen en el glosario que sigue a esta introducción (ver Tabla 2). Por ejemplo, **HTTP** (Protocolo de Transferencia de Hipertexto) es el conjunto de reglas con el que dos programas se envían mensajes por una red, el mismo que usa un navegador para abrir una página web; SITRA lo usa para que la billetera le envíe sus solicitudes.

El documento sigue la plantilla oficial del curso (Corporación Universitaria Remington, 2026), que es una versión simplificada de arc42, una plantilla abierta para documentar arquitecturas (Starke y Hruschka, s.f.). Las vistas se dibujan con el modelo C4, que representa un sistema en niveles de detalle, como un mapa que se amplía (Brown, s.f.), y con UML (Lenguaje Unificado de Modelado), la notación gráfica estándar para software. Los atributos de calidad se describen con el modelo de la norma ISO/IEC 25010 (International Organization for Standardization [ISO], 2011) y se miden con la ISO/IEC 25023 (ISO, 2016). Las decisiones se registran como ADR, registros cortos de decisiones de arquitectura (Nygard, 2011), y la evaluación final usa los conceptos de compromiso y punto de sensibilidad del método ATAM (Bass et al., 2021; Kazman et al., 2000).

En la parte técnica, la aplicación se construyó con el marco de trabajo Spring Boot (Spring, 2024) y la base de datos MariaDB (MariaDB plc, s.f.). Cada petición se autentica con un token JWT (Jones et al., 2015) firmado con el algoritmo HMAC (Krawczyk et al., 1997); los secretos de los clientes se guardan con bcrypt (Provos y Mazières, 1999), y los códigos de respuesta siguen el significado que les da el estándar HTTP (Fielding et al., 2022). El tratamiento de los datos personales de los asociados se rige por la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).

El documento tiene seis secciones: los factores que condicionan la arquitectura (Sección 1), la estrategia y la tecnología (Sección 2), cinco vistas o diagramas de la arquitectura (Sección 3), las decisiones de arquitectura (Sección 4), el programa ejecutable con sus pruebas (Sección 5) y la evaluación de compromisos y riesgos (Sección 6). Al final están las conclusiones, las referencias y dos anexos: la lista de chequeo de entrega (Anexo A) y las correcciones de esta versión (Anexo B).

Control de versiones del documento

La Tabla 1 resume cómo evolucionó el documento. La columna “Elaboración” indica cuándo se escribió cada versión y la columna “Subida a GitHub” cuándo llegó al repositorio en línea. Las dos fechas no siempre coinciden: la versión 0.5 se presentó en clase el 24 de octubre desde el equipo local, pero se subió al repositorio el 6 de noviembre, junto con las versiones siguientes.

Tabla 1

Historial de versiones del documento

Versión	Elaboración	Subida a GitHub	Contenido	Clase
0.1	26/09/2026	26/09/2026	Ficha de drivers revisada con las restricciones del cliente y Sección 1 completa: visión del negocio, alcance, historias de usuario, atributos de calidad con métricas y restricciones.	Clase 1
0.4	16/10/2026	16/10/2026	Sección 2 (patrón N-Capas y pila tecnológica), vistas de contexto y contenedores (3.1 y 3.2), vistas de clases y secuencia (3.3 y 3.4) y script SQL de la base de datos (5.2). Se ajustó en 1.3 el nombre y la fórmula de la métrica principal.	Clases 2, 3 y 4 (entregadas juntas en la clase del 17/10)
0.5	24/10/2026	06/11/2026	Sección 4 con ADR-001 a ADR-003, vista de despliegue (3.5), arranque del corte vertical y cambio de PostgreSQL a MariaDB (ADR-002). El modelo incorpora al asociado, los intentos rechazados y la IP de origen pedidos en la Clase 4.	Clase 5
0.6	06/11/2026	06/11/2026	CRUD completo de Transferencia con token JWT y BCrypt (ADR-004 y ADR-005), consulta de saldo (HU-02) y consulta por fechas con verificación de integridad de la bitácora (HU-03).	Clase 6
1.0	06/11/2026	06/11/2026	Pruebas de la arquitectura (5.4), análisis de compromisos, puntos de sensibilidad y conclusión (Sección 6). Documento completo, adelantado a las clases 7 y 8.	Clases 7 y 8 (adelantadas)

Versión	Elaboración	Subida a GitHub	Contenido	Clase
1.1	07/11/2026	07/11/2026	Corrige los 12 hallazgos de la revisión de trazabilidad (H-01 a H-12, ver Anexo B) y aplica el formato APA 7: portada, resumen, glosario, tablas y figuras numeradas, términos explicados y referencias.	Revisión de la docente

Nota. Las fechas de subida corresponden a los commits del repositorio https://github.com/JorgeMario-V/sitra.

Glosario de términos técnicos

La Tabla 2 explica, en palabras sencillas y en orden alfabético, los términos técnicos y las siglas que se usan en el documento, y dice para qué se usa cada uno dentro de SITRA. Quien no tenga formación en programación puede leer primero esta tabla y volver a ella cuando encuentre un término desconocido.

Tabla 2

Términos técnicos usados en el documento

Término	Qué significa	Para qué se usa en SITRA
ADR (registro de decisión de arquitectura)	Documento corto que registra una decisión importante del diseño: el problema, las alternativas, lo que se decidió y lo que se sacrifica. No se edita: si la decisión cambia, se escribe uno nuevo que lo reemplaza.	Hay cinco (ADR-001 a ADR-005) en la Sección 4 y en la carpeta docs/adr/ del repositorio.
API (interfaz de programación de aplicaciones)	La “puerta” por la que un programa le pide datos o acciones a otro, siguiendo reglas fijas. Es como la ventanilla de un banco: el cliente no entra a la bóveda, solo hace solicitudes por la ventanilla.	SITRA es una API: la billetera le pide saldos y transferencias, y nunca toca el núcleo directamente.
API REST	Estilo de API en el que cada recurso tiene una dirección web y se usan las operaciones de HTTP: GET para consultar, POST para crear, PUT para modificar y DELETE para eliminar o anular.	Las rutas de SITRA, como /api/v1/transferencias, siguen este estilo (Tabla 13).

Término	Qué significa	Para qué se usa en SITRA
Base de datos relacional	Programa que guarda la información en tablas relacionadas entre sí y garantiza reglas de integridad (por ejemplo, que no existan dos transferencias con el mismo código).	SITRA usa MariaDB para guardar asociados, cuentas, transferencias y la bitácora.
Billetera digital	Aplicación de pagos de una empresa externa a la cooperativa, con la que las personas consultan saldos y envían dinero desde el celular.	Es el principal usuario de SITRA: hace las transferencias (HU-01) y consulta saldos (HU-02).
Bitácora de auditoría	Registro histórico de todas las operaciones: quién hizo qué, sobre qué cuenta, con qué resultado, desde dónde y cuándo.	Es la tabla registro_auditoria. No se puede modificar ni borrar, y es la evidencia que exige el regulador.
Borrado lógico	En lugar de eliminar un registro, se marca como inactivo. El dato sigue existiendo y se puede consultar después.	Anular una transferencia la marca como ANULADA y con activo = falso; nunca se borra la fila.
BCrypt	Método para guardar contraseñas como una huella irreversible y lenta de calcular a propósito, para que un atacante no pueda adivinarlas probando millones de combinaciones.	Los secretos de la billetera y del auditor solo se guardan como huella BCrypt (ADR-005).
C4 (modelo)	Forma de dibujar la arquitectura en cuatro niveles de detalle: contexto, contenedores, componentes y código, como un mapa que se va ampliando.	Las vistas 3.1 (contexto) y 3.2 (contenedores) siguen este modelo.
Código de estado HTTP	Número de tres cifras con el que una API responde si la solicitud funcionó o por qué falló; por ejemplo, 201 significa “creado” y 401 “no autorizado”.	Cada respuesta de SITRA lleva su código; la Tabla 14 explica los que se usan.
Commit, Git y GitHub	Git es un programa que guarda la historia de cambios de los archivos de un proyecto; cada cambio guardado con su descripción es un commit. GitHub es el sitio web donde se publica esa historia.	Todo el proyecto está en https://github.com/JorgeMario-V/sitra con su historial de commits (5.1).
Corte vertical	Una sola funcionalidad completa que atraviesa todas las capas del sistema, desde la petición hasta la base de datos y de regreso. Sirve para comprobar temprano que la arquitectura funciona.	El corte vertical de SITRA es la HU-01: registrar una transferencia (Sección 5).
CRUD	Las cuatro operaciones básicas sobre un dato: crear (Create), consultar (Read), actualizar (Update) y eliminar (Delete).	SITRA implementa el CRUD completo de Transferencia (5.3).

Término	Qué significa	Para qué se usa en SITRA
Deadlock (bloqueo mutuo)	Situación en la que dos operaciones simultáneas esperan cada una a que la otra termine. La base de datos la detecta, cancela una de las dos y la deshace por completo.	Puede ocurrir cuando varias operaciones escriben en la bitácora al mismo tiempo (riesgo R-03).
Endpoint	Cada dirección de la API que atiende un tipo de solicitud, por ejemplo GET /api/v1/cuentas/AH-1001/saldo.	SITRA tiene nueve (Tabla 13).
Framework (marco de trabajo)	Conjunto de herramientas y reglas ya construidas sobre las que se escribe un programa, para no empezar de cero.	SITRA usa Spring Boot.
Hash o huella (SHA-256)	Resumen matemático de un dato: siempre tiene el mismo largo, y si el dato cambia en un solo carácter la huella cambia por completo. No se puede volver de la huella al dato original.	Cada fila de la bitácora guarda su huella encadenada con la anterior; así se detecta cualquier alteración.
HMAC	Forma de firmar un mensaje con una clave secreta: quien no tiene la clave no puede crear una firma válida ni modificar el mensaje sin que se note.	Con HMAC-SHA256 se firman los tokens JWT de SITRA.
HTTP y HTTPS	HTTP (Protocolo de Transferencia de Hipertexto) son las reglas con las que dos programas se envían mensajes por la red. HTTPS es la misma comunicación, pero cifrada para que nadie pueda leerla en el camino.	La billetera y el auditor hablan con SITRA por HTTP en el laboratorio; en producción se usará HTTPS (riesgo R-05).
Idempotencia	Propiedad de una operación que, si se repite por error, no produce un efecto doble.	Si la billetera reenvía una transferencia con el mismo código de operación, SITRA no la cobra dos veces (responde 409).
ISO/IEC 25010 y 25023	Normas internacionales de calidad del software: la 25010 define las características de calidad (seguridad, fiabilidad, mantenibilidad…) y la 25023 explica cómo medirlas.	Los atributos de calidad de la Sección 1.3 y su métrica principal salen de estas normas.
JAR	Archivo único que empaqueta un programa Java con todo lo que necesita para ejecutarse con el comando java -jar.	SITRA se despliega como un solo JAR (3.5).
JPA, ORM e Hibernate	Un ORM (mapeo objeto-relacional) traduce automáticamente entre las clases del programa y las tablas de la base de datos. JPA es el estándar de Java para hacerlo e Hibernate es la herramienta que lo implementa.	Las clases del paquete model corresponden una a una con las tablas (Tabla 9).

Término	Qué significa	Para qué se usa en SITRA
JSON	Formato de texto sencillo para intercambiar datos entre programas, con pares nombre-valor, por ejemplo {"monto": 300000}.	Las solicitudes y respuestas de la API viajan en JSON.
JUnit y Mockito	JUnit es una herramienta para escribir pruebas automáticas en Java. Mockito crea “dobles” de prueba: piezas falsas que imitan a las reales, para probar una parte sin depender de las demás.	Hay 21 pruebas JUnit; las unitarias usan Mockito para imitar el núcleo y la base de datos (5.4).
JWT (token)	JSON Web Token: credencial digital firmada que el cliente presenta en cada petición. Dice quién es, qué rol tiene y cuándo vence; si alguien la modifica, la firma deja de coincidir.	La billetera y el auditor obtienen un token de 30 minutos y lo envían en cada solicitud (ADR-005).
Llave primaria, llave foránea, CHECK, UNIQUE e índice	Reglas de una base de datos: la llave primaria identifica cada fila; la foránea obliga a que un dato exista en otra tabla; CHECK exige una condición (monto mayor que cero); UNIQUE impide repetidos; el índice acelera las búsquedas.	Todas se usan en el script de la Sección 5.2 (Tabla 12).
MariaDB, XAMPP y HeidiSQL	MariaDB es un motor de base de datos gratuito. XAMPP es un paquete gratuito que lo instala junto con otras herramientas. HeidiSQL es un programa para consultar la base con ventanas.	Ya están instalados en los equipos disponibles; por eso se eligieron (ADR-002).
Maven y Maven Wrapper	Maven es la herramienta que descarga las librerías de un proyecto Java, lo compila y ejecuta sus pruebas. El Maven Wrapper (mvnw.cmd) es un pequeño script que descarga Maven automáticamente, para no tener que instalarlo.	Con mvnw.cmd se arranca la aplicación y se corren las pruebas sin instalar nada adicional (5.3).
Núcleo transaccional legado	El sistema central y antiguo de la cooperativa que lleva las cuentas y mueve el dinero. “Legado” significa que sigue en uso porque es crítico, aunque sea viejo y difícil de cambiar.	SITRA no lo modifica: solo se conecta a él por medio del conector (ADR-001).
Patrón arquitectónico y N-Capas	Un patrón arquitectónico es una forma probada de organizar un sistema. En N-Capas el sistema se divide en niveles apilados (presentación, negocio, datos) y cada nivel solo habla con el de abajo.	Es el patrón de SITRA (Sección 2.1 y ADR-001).
Postman	Programa para enviar solicitudes a una API y revisar sus respuestas; permite guardar colecciones de solicitudes con pruebas automáticas.	La colección de 19 solicitudes y 30 pruebas demuestra el corte vertical (5.4).

Término	Qué significa	Para qué se usa en SITRA
Puerto	Número que identifica a un programa dentro de un computador en la red, como el número de apartamento dentro de un edificio.	La API escucha en el puerto 8080 y la base de datos en el 3306 (3.5).
SQL y DDL	SQL es el lenguaje para trabajar con bases de datos relacionales. DDL es la parte de SQL que crea la estructura: tablas, columnas y reglas.	El esquema de SITRA está en los scripts SQL V1 a V4 (5.2).
Spring Boot	Framework para construir aplicaciones Java que ya trae servidor web, conexión a bases de datos y configuración por variables.	Es la base de la API de SITRA.
Trade-off (compromiso)	Lo que se gana y lo que se sacrifica al tomar una decisión. Toda decisión de arquitectura tiene uno.	La Sección 6.1 los analiza.
Punto de sensibilidad	Parte de la arquitectura que, si cambia o falla, afecta directamente un atributo de calidad.	La Sección 6.2 identifica siete, con su riesgo y mitigación.
Transacción	Conjunto de operaciones de la base de datos que se ejecuta completo o no se ejecuta: nunca queda a medias.	Una transferencia y su registro en la bitácora se guardan en la misma transacción.
Trigger (disparador)	Regla guardada dentro de la base de datos que se ejecuta sola cuando ocurre algo, por ejemplo al insertar una fila.	Los triggers escriben la bitácora y bloquean su modificación o borrado (ADR-003).
UML	Lenguaje Unificado de Modelado: notación gráfica estándar para dibujar software (clases, secuencias, despliegue).	Las figuras de la Sección 3 usan UML.
Variable de entorno	Valor de configuración que se le entrega a un programa desde el sistema operativo, fuera del código. Así las contraseñas no quedan escritas en los archivos del proyecto.	Las siete variables SITRA_* configuran puertos, base de datos y secretos (Tabla 10).

Sección 1. Introducción y drivers arquitectónicos

Los **drivers arquitectónicos** son los factores que obligan a que la arquitectura sea como es: los requisitos funcionales más importantes, los atributos de calidad que el sistema debe cumplir y las restricciones que no se pueden discutir. Esta sección los presenta.

1.1 Visión general del negocio y alcance

Problema de negocio

La cooperativa opera sobre un núcleo transaccional construido hace 20 años en un lenguaje propietario, es decir, un lenguaje que pertenece a una empresa y no es de uso libre. Su documentación se perdió y nadie del equipo actual lo escribió. El regulador le exige, en un plazo fijo de seis meses, integrarse con una billetera digital externa y entregar trazabilidad de sus transacciones; si no cumple, arriesga la licencia de operación.

Hoy el problema lo padecen el área de cumplimiento y la gerencia. Sin software, la evidencia para el auditor se reconstruye a mano con reportes parciales del núcleo y conciliaciones en hojas de cálculo: un proceso lento, que no demuestra que los datos no fueron alterados y que no alcanza los tiempos que pide el auditor. La posibilidad de que un tercero, como la billetera, use funciones del núcleo simplemente no existe.

Además, el cliente fijó tres condiciones que no se discuten: el núcleo no se va a cambiar, porque lo seguirán usando y su negocio no es el software; no hay dinero adicional ni se invertirá en productos comerciales de API, de modo que solo se trabaja con lo que se tiene; y el personal de la cooperativa se dedica a sus funciones, no a desarrollar ni mantener software.

Propuesta de solución

El sistema permite a la billetera digital externa, previamente autenticada, consultar saldos y registrar transferencias sobre cuentas autorizadas de la cooperativa, para entregar al auditor el 100 % de esas operaciones con evidencia verificable de que no fueron alteradas, sin modificar el núcleo legado y sin inversión adicional.

Alcance

Lo que el proyecto incluye es lo siguiente:

- Un conector externo al núcleo que usa solo lo que el núcleo ya tiene hoy (sus salidas e interfaces actuales); no se compra ni se instala ningún producto.
- Dos operaciones para la billetera, expuestas por medio de una API: consulta de saldo y registro de transferencias, con su corrección, reintento y anulación.
- Autenticación de quien llama con un token JWT y autorización por rol: la billetera (BILLETERA) y el auditor (AUDITOR) solo pueden hacer lo que les corresponde.
- Una lista blanca de cuentas autorizadas: solo esas cuentas son visibles para el tercero; el resto de cuentas del núcleo queda oculto.
- Una bitácora de auditoría que no se puede modificar ni borrar, que el auditor puede consultar y verificar.
- Un simulador del núcleo, para poder probar todo sin tocar el sistema real, del que no existe ambiente de pruebas.

Lo que queda fuera del alcance es lo siguiente:

- Modificar, reescribir o reemplazar el núcleo legado. Tampoco se plantea una migración gradual: el núcleo se queda indefinidamente.
- Cualquier otra función del núcleo distinta de saldo y transferencias.
- Una aplicación móvil o una pantalla para el usuario final: quien usa SITRA es la billetera, que es otro sistema.
- Productos o licencias de pago: puertas de enlace de API comerciales, servicios de nube facturados o herramientas de auditoría de pago.
- Alta disponibilidad (varias copias del sistema funcionando a la vez) y reportes o tableros analíticos avanzados.

- Contratar soporte externo: no es una decisión de arquitectura y se eleva a la junta directiva (ver 1.4).

1.2 Historias de usuario arquitectónicamente significativas

Una **historia de usuario** describe una necesidad con la fórmula “Como [quién], quiero [qué], para [para qué]”. Es **arquitectónicamente significativa** cuando, al construirla, pone a prueba las partes más críticas o riesgosas del sistema y no solo las más simples. La Tabla 3 presenta las tres historias elegidas, por qué son significativas y cómo se comprueba que funcionan.

Tabla 3

Historias de usuario arquitectónicamente significativas

ID	Historia de usuario	Por qué es arquitectónicamente significativa	Cómo se comprueba que funciona
HU-01	Como billetera digital externa autenticada, quiero registrar una transferencia sobre una cuenta autorizada, para que se ejecute en el núcleo y quede con evidencia auditable.	Es la más riesgosa: recorre todas las capas y pone a prueba a la vez la autenticación, la validación de datos, la integración con el núcleo legado, el almacenamiento y la bitácora. Por eso es el corte vertical (Sección 5).	La solicitud POST /api/v1/transferencias responde 201 y la operación aparece en la bitácora con su huella. Lo prueban las peticiones 04 a 15 de Postman y la prueba de integración JUnit.
HU-02	Como billetera digital externa autenticada, quiero consultar el saldo de una cuenta autorizada, para mostrárselo al usuario final.	Pone a prueba la integración de solo lectura con el núcleo y el mínimo privilegio, porque solo se ven las cuentas de la lista blanca. Además valida que el saldo no se copia: siempre se lee del núcleo, que es la única fuente de verdad.	GET /api/v1/cuentas/AH-1001/saldo responde 200 con el saldo leído del núcleo, y una cuenta fuera de la lista blanca responde 404. Lo prueba la petición 08 de Postman.
HU-03	Como auditor, quiero consultar las operaciones por cuenta y rango de fechas y verificar su integridad, para entregar la evidencia que exige el regulador.	Materializa el atributo crítico del caso, la trazabilidad. Pone a prueba el control por rol (solo el auditor la ve) y el mecanismo que demuestra que un registro no fue alterado.	La consulta responde 200 en menos de 5 segundos, la verificación responde íntegra = verdadero y la billetera recibe 403. Lo prueban las peticiones 17 a 19 de Postman.

Nota. Las tres historias validan el patrón, la seguridad, el almacenamiento y la integración con un sistema externo. Se descartaron historias como “ver el estado del servicio” o “listar cuentas” porque no ponen a prueba ninguna de esas partes.

1.3 Atributos de calidad prioritarios

Un **atributo de calidad** describe qué tan bien debe hacer el sistema su trabajo (qué tan seguro, confiable o fácil de mantener debe ser), no qué hace. Se toman del modelo de la norma ISO/IEC 25010 (ISO, 2011) y cada uno se especifica con un **escenario de calidad**: un estímulo (lo que pasa), el entorno (en qué condiciones), la respuesta esperada y una medida que permite comprobarla. La prioridad la da el caso: la seguridad y la trazabilidad están “por encima de cualquier otro atributo”; la fiabilidad sigue porque se mueve dinero de los asociados, y la mantenibilidad sube a primer orden porque no habrá personal técnico que sostenga el sistema. La Tabla 4 presenta los tres atributos.

Tabla 4

Atributos de calidad prioritarios (ISO/IEC 25010) y su especificación cuantitativa

ID	Atributo (ISO 25010)	Escenario de calidad (estímulo, entorno y respuesta)	Métrica objetivo (medible)	Cómo se verifica
AC-01	Seguridad: responsabilidad, no repudio e integridad (en la ficha de drivers: AC-06).	La billetera hace operaciones en condiciones normales; además, un cliente sin token intenta registrar una transferencia y alguien intenta modificar un registro de la bitácora directamente en la base de datos.	Métrica principal, completitud del registro de auditoría (ISO/IEC 25023): X = A / B, donde A son las operaciones registradas en la bitácora y B las operaciones realizadas. Meta: X = 1, es decir, el 100 %. Además: 0 registros de la bitácora modificables o borrables; respuesta 401 en el 100 % de los intentos sin token; la consulta del auditor por cuenta y fechas responde en 5 segundos o menos.	Postman: peticiones 03 (401), 18 (todo quedó registrado y respondió en menos de 5 s) y 19 (cadena íntegra). Prueba de integración JUnit (mvn-test-integracion.txt). Script demo_sustentacion.sql: los cambios y borrados sobre la bitácora son rechazados (resultado_verificacion_mariadb.txt).
AC-02	Fiabilidad: tolerancia a fallos (en la ficha: AC-05).	El núcleo legado no responde mientras se procesa una transferencia ya aceptada. El sistema debe cerrarla sin dejarla a medias.	El 100 % de estas transferencias termina en estado FALLIDA con respuesta 503, sin efecto parcial. Repetir la misma referencia responde 409 y nunca duplica la operación. Meta pendiente: espera máxima de 5 segundos al núcleo real. Todavía no está implementada (deuda técnica del ADR-001); hoy se verifica el resultado (FALLIDA, 503, sin efecto parcial), no el tiempo.	Pruebas unitarias con el núcleo imitado por Mockito: núcleo sin respuesta → FALLIDA; código repetido → 409 (mvn-test.txt). Modo sin-respuesta del simulador y petición 05 de Postman (409).
AC-03	Mantenibilidad: modularidad y capacidad de ser probado (en la ficha: AC-07).	Se necesita probar sin el núcleo real, porque no hay ambiente de pruebas, o cambiar la forma de conectarse a él, y no hay personal técnico que conozca el núcleo.	El cambio toca como máximo 1 clase del conector y 1 valor de configuración, y 0 archivos de las capas de presentación, negocio y datos. Todas las pruebas corren sin conexión al núcleo real.	Revisión del código: el conector se elige con la variable SITRA_NUCLEO_MODO. mvnw.cmd test corre las 17 pruebas unitarias sin núcleo y sin base de datos (mvn-test.txt).

Nota. Los archivos citados en la última columna están en evidencias/entrega-07-pruebas-y-tradeoffs/ del repositorio y se generan con scripts\generar_evidencias.cmd.

La trazabilidad no es una característica propia de la ISO/IEC 25010. Se ubica dentro de la seguridad, en las subcaracterísticas de responsabilidad (poder saber quién hizo cada cosa) y no repudio (que nadie pueda negar después lo que hizo). La métrica “completitud del registro de auditoría” está definida para esa subcaracterística en la ISO/IEC 25023, la norma que explica cómo medir la 25010 (ISO, 2016).

1.4 Restricciones

Las restricciones vienen dadas por el cliente y el regulador; la arquitectura las acata y no las discute. La Tabla 5 las presenta con su origen.

Tabla 5

Restricciones del proyecto

Tipo	Restricción	Origen y justificación
Técnica (RT-01)	El núcleo legado no se modifica, ni ahora ni a futuro. Solo se consumen los puntos de acceso que ya expone.	El núcleo tiene 20 años, está en un lenguaje propietario y no tiene documentación: tocarlo tiene un riesgo muy alto de dañar algo que nadie sabe reparar. Además, el cliente dijo explícitamente que no va a cambiar su sistema.
Técnica (RT-02)	No existe ambiente de pruebas del núcleo; toda prueba previa se hace contra un sustituto controlado (el simulador).	Es consecuencia de la restricción anterior y de la falta de presupuesto: no hay con qué montar una copia del núcleo para hacer pruebas sin riesgo.
Técnica (RT-03)	La base de datos debe ser relacional, de licencia libre y con transacciones; la comunicación con el tercero es una API REST con mensajes JSON sobre HTTPS.	Es un lineamiento del curso, y además la bitácora exige reglas de integridad que solo garantiza una base relacional. REST y JSON son el estándar que entiende cualquier billetera.

Tipo	Restricción	Origen y justificación
Operativa (RA-02)	Cero presupuesto: solo software libre e infraestructura que la cooperativa ya tiene. Nada de licencias, nube de pago ni productos comerciales de API.	Lo dijo la gerencia: “solo tenemos lo que se tiene” y no se invertirá en APIs. Toda tecnología elegida debe ser gratuita y funcionar en los equipos existentes.
Operativa (RA-03)	Cero personal técnico: la solución debe operarse por configuración y arrancar sola, sin nadie que la desarrolle o la mantenga.	Lo dijo el cliente: el personal solo se dedica a las funciones de la cooperativa y no queda nadie del equipo original. Por eso se prefiere lo más estándar y simple de mantener.
De tiempo	Plazo regulatorio fijo de 6 meses. En lo académico: corte vertical funcionando el 07/11/2026 y documento final el 21/11/2026.	El plazo lo fija el regulador, que puede retirar la licencia de operación; las fechas académicas las fija el cronograma del curso.
Normativa	Trazabilidad auditable y registros que no se puedan alterar; datos personales tratados según la Ley 1581 de 2012 (protección de datos personales o habeas data).	Es la exigencia del auditor y el marco legal colombiano. Por eso SITRA guarda solo los datos mínimos del asociado y deja en la bitácora quién consultó qué.

Nota. RT significa restricción técnica y RA restricción administrativa, como en la ficha de drivers de la Clase 1.

Existe una **tensión declarada** entre las restricciones: el regulador obliga a conectar el núcleo con la billetera, pero el cliente no da dinero ni personal, y con literalmente nada nuevo no se puede cumplir. La arquitectura propone el mínimo indispensable —usar primero lo que el núcleo ya tiene y, donde no alcance, un conector pequeño con software gratuito sobre los equipos existentes— y deja en manos de la junta directiva quién lo opera y lo sostiene.

Por esa razón, antes de diseñar se formuló la siguiente pregunta al cliente (punto 5 de la ficha de drivers): “Si no hay presupuesto ni personal que desarrolle o mantenga la integración, ¿quién la va a operar y sostener una vez esté en funcionamiento, y con qué recursos que la cooperativa ya tenga?”. La pregunta revela un supuesto oculto del caso: se da por hecho que alguien construirá y sostendrá la integración, pero con cero presupuesto (RA-02) y cero personal

técnico (RA-03) ese alguien no existe. Esa decisión le corresponde a la junta directiva, no al arquitecto.

Sección 2. Estrategia arquitectónica y tecnológica

2.1 Patrón arquitectónico principal: N-Capas

Un **patrón arquitectónico** es una forma probada de organizar un sistema. Se selecciona el patrón **N-Capas**, en el que el sistema se divide en niveles apilados —presentación, negocio y datos, más un conector del núcleo— y cada nivel solo habla con el que está debajo. Se elige porque el atributo prioritario AC-03 (mantenibilidad) exige separar responsabilidades y poder cambiar la conexión con el núcleo por un simulador sin tocar la lógica de negocio, y porque la restricción de cero personal técnico exige una estructura que cualquier programador de Java reconozca sin capacitación.

Se descarta el patrón Hexagonal porque su costo en número de clases, interfaces y curva de aprendizaje contradice esa misma restricción de cero personal técnico; de él solo se toma una idea: el núcleo se usa a través de una única interfaz, NucleoConector. Se descarta Microservicios porque su complejidad de operación contradice las restricciones de cero presupuesto y cero personal. La Tabla 6 compara los cuatro patrones candidatos.

Tabla 6

Comparación de los patrones arquitectónicos candidatos

Patrón candidato	A favor en este caso	En contra en este caso	Veredicto
MVC monolítico (Modelo-Vista-Controlador en una sola aplicación)	Es el más simple y rápido de construir: con un controlador por recurso se tendría la HU-01 funcionando en pocos días, dentro del plazo de seis meses.	El controlador terminaría hablando directamente con el núcleo y con la base de datos. No hay un lugar natural para poner el simulador del núcleo: para probar sin el núcleo real habría que modificar el código de negocio, y eso es justo lo que la métrica de AC-03 prohíbe. Con el tiempo, cada controlador acumula validaciones, llamadas al núcleo y consultas, y se vuelve difícil de entender para quien llegue sin conocer el sistema.	Descartado: incumple la métrica de AC-03 (cambiar el núcleo no debe tocar el negocio).
N-Capas	Cada capa tiene una sola responsabilidad: la presentación recibe las peticiones, el negocio aplica las reglas y los datos guardan la información. Es la estructura estándar de Spring Boot, la que muestra cualquier tutorial o curso de Java, así que quien herede el sistema la reconoce sin capacitación. El conector del núcleo queda aislado en su propia capa y se cambia por configuración.	Exige escribir más archivos que un MVC: por cada operación hay un controlador, un servicio y un repositorio, en lugar de un solo controlador que lo hace todo. En SITRA esto se traduce en 45 archivos Java (clases, interfaces y enumeraciones) para cuatro recursos, bastantes más de los que necesitaría un MVC simple. A cambio, cada archivo es corto y hace una sola cosa, y para cambiar una regla de negocio se sabe exactamente qué archivo abrir. También hay un pequeño costo en tiempo de respuesta por pasar por varias capas, que no se nota con el volumen de una cooperativa.	Seleccionado, con un conector intercambiable para el núcleo (ADR-001).
Hexagonal (puertos y adaptadores)	Da la máxima independencia del núcleo y de la base de datos: el negocio no conoce ninguna tecnología, así que se puede probar todo con piezas imitadas.	Introduce muchos más conceptos y archivos: puertos de entrada y de salida, adaptadores para cada tecnología y traducciones entre modelos. Para dos operaciones es una estructura demasiado elaborada, y sin personal técnico nadie podría sostenerla ni explicarla a un recién llegado.	Descartado como patrón completo; se toma solo la idea de la interfaz del conector.

Patrón candidato	A favor en este caso	En contra en este caso	Veredicto
Microservicios	Permite escalar y desplegar cada parte por separado, por ejemplo la consulta de saldo independiente de las transferencias.	Serían varios servicios que instalar, conectar, vigilar y actualizar, cada uno con su propio despliegue, más una red entre ellos que puede fallar. Exige personal de operación y, normalmente, infraestructura de pago. Ninguna de las dos cosas existe en este caso.	Descartado: contradice RA-02 (cero presupuesto) y RA-03 (cero personal).

Capas y responsabilidades

La Tabla 7 muestra cada capa tal como quedó en el código: qué responsabilidad tiene, qué clases la forman y qué hace cuando la billetera registra una transferencia.

Tabla 7

Capas de SITRA y sus responsabilidades

Capa	Responsabilidad	Clases principales (paquete)	Qué hace cuando llega una transferencia (HU-01)
Seguridad	Rechazar toda petición a /api/ sin un token válido (401) antes de que llegue a los controladores; emitir los tokens y comparar los secretos con BCrypt.	FiltroJwt, JwtServicio y ConfiguracionSeguridad (paquete security).	Revisa que el token esté presente, firmado con el secreto correcto y sin vencer, y anota quién es el cliente para las capas siguientes.
Presentación	Recibir la petición HTTP y devolver el código de respuesta correcto. No contiene reglas de negocio. Los errores se convierten en respuestas en un solo lugar.	AuthController, TransferenciaController, SaldoController y AuditoriaController (controller); ManejadorErrores (exception).	Recibe el JSON de la transferencia, se lo pasa al servicio y traduce el resultado a 201, 422 o 503.
Negocio	Aplicar las reglas: rol del cliente (403), cuenta autorizada, monto positivo, código no repetido, estados de la transferencia, reverso al anular y registro de los rechazos en la bitácora.	TransferenciaService, CuentaService, AuditoriaService e IdentificacionService (service).	Valida los datos, llama al núcleo por medio del conector y decide si la transferencia queda CONFIRMADA, RECHAZADA o FALLIDA.

Capa	Responsabilidad	Clases principales (paquete)	Qué hace cuando llega una transferencia (HU-01)
Datos	Guardar y leer de MariaDB con JPA; ejecutar la consulta que recalcula las huellas de la bitácora.	TransferenciaRepository y demás repositorios (repository); entidades (model).	Guarda la transferencia con su estado final; en ese momento el trigger de la base escribe la bitácora.
Conector del núcleo	Ser la única parte que habla con el núcleo legado. Tiene dos versiones intercambiables por configuración: la real y el simulador.	NucleoConector (interfaz), NucleoSimuladoConector y NucleoLegadoConector (integration).	Pide al núcleo ejecutar el débito y devuelve el comprobante, el motivo del rechazo o el aviso de que el núcleo no respondió.

Nota. Regla de dependencias: cada capa solo llama a la de abajo. El servicio no sabe si habla con el núcleo real o con el simulador; solo conoce la interfaz NucleoConector.

2.2 Pila tecnológica

La **pila tecnológica** es el conjunto de herramientas con que se construye el sistema. La Tabla 8 presenta cada una, qué es, qué versión se usa, qué alternativa se descartó y por qué se eligió, ligando cada elección a un atributo de calidad (AC) o a una restricción (RT o RA).

Tabla 8

Pila tecnológica

Capa	Tecnología elegida (qué es)	Versión	Alternativa descartada	Justificación
Lenguaje	Java (OpenJDK): lenguaje de programación de uso general, gratuito.	21, versión de soporte extendido (LTS)	Node.js 20 (JavaScript en el servidor)	Restricción de tiempo: el estudiante ya domina Java y Spring Boot. Es gratuito (RA-02) y la versión LTS recibe actualizaciones durante años, lo que reduce el mantenimiento (AC-03).

Capa	Tecnología elegida (qué es)	Versión	Alternativa descartada	Justificación
Framework	Spring Boot: marco de trabajo para aplicaciones Java que trae servidor web, acceso a datos y configuración.	3.3.5	Express (Node.js) o Quarkus (Java)	AC-03: impone la estructura por capas con anotaciones conocidas (@RestController, @Service, @Repository) y permite cambiar el conector por el simulador con una propiedad (sitra.nucleo.modo), sin tocar código.
Seguridad	Token JWT firmado con HMAC-SHA256 (incluido en Java) y secretos guardados con BCrypt (módulo spring-security-crypto).	JDK 21 y spring-security-crypto 6.3	Sesiones en el servidor; servidor de identidad como Keycloak	ADR-005. AC-01: quien llama es otro sistema, así que cada petición debe probar quién la hace, y ese dato queda en la bitácora. No se instala ninguna plataforma nueva (RA-02 y RA-03).
Acceso a datos	Spring Data JPA con Hibernate: traduce entre clases Java y tablas sin escribir SQL repetitivo.	Spring Data 3.3 e Hibernate 6.5	JDBC con SQL escrito a mano	AC-03: los repositorios se escriben con nombres de métodos y las entidades corresponden una a una con las tablas. Se acepta perder control fino del SQL (ver 6.1).
Migraciones	Scripts SQL numerados V1 a V4, que se ejecutan en orden con la consola de MySQL o con HeidiSQL.	4 scripts (V1 a V4) y uno que los une, instalar_todo.sql	Que Hibernate cree las tablas solo (ddl-auto) o la herramienta Flyway	AC-01: el esquema queda versionado en Git y se puede auditar cuándo cambió. Los triggers y el procedimiento no los puede crear Hibernate. La consola y HeidiSQL ya vienen con XAMPP (RA-03).
Base de datos	MariaDB: motor de base de datos relacional gratuito, incluido en el paquete XAMPP.	10.4 o superior (también se probó en 10.11)	PostgreSQL 16 (se usó hasta la Clase 4) y MongoDB	ADR-002. AC-01: tiene llaves foráneas, CHECK, UNIQUE, transacciones y triggers que impiden modificar la bitácora. Ya está instalada en los equipos (RA-02 y RA-03).
Huella de integridad	Función SHA2(…, 256) de MariaDB, calculada por un trigger en cada fila nueva de la bitácora.	Incluida en MariaDB 10.4+	Un servicio de auditoría en la nube	ADR-003. Viene con la base de datos y no cuesta nada (RA-02). Permite detectar si un registro fue alterado sin depender del código de la aplicación.
Conector del núcleo	Interfaz Java propia (NucleoConector) con un simulador para pruebas y un conector real para producción.	Propia del proyecto (sin librerías externas)	Puerta de enlace de API comercial	No hay licencias que pagar (RA-02) y no existe ambiente de pruebas del núcleo (RT-02): el simulador permite probar todo sin tocar producción.

Capa	Tecnología elegida (qué es)	Versión	Alternativa descartada	Justificación
Interfaz de usuario	No se construye pantalla: el consumidor es la billetera, otro sistema; el auditor usa la API o HeidiSQL.	No aplica	Una aplicación web en React	Está fuera del alcance (1.1). Postman demuestra el flujo completo sin necesidad de pantallas.
Pruebas	JUnit 5 y Mockito para pruebas automáticas en Java; MockMvc para probar la API completa; Postman para probarla desde afuera.	JUnit 5.10, Mockito 5 y Postman (versión gratuita)	Solo pruebas manuales	Permiten verificar con evidencia repetible las métricas de AC-01, AC-02 y AC-03 (5.4).
Construcción	Maven Wrapper (mvnw.cmd): descarga Maven automáticamente para compilar, ejecutar y probar.	Wrapper 3.3.2 con Maven 3.9.9	Exigir instalar Maven a mano	Criterio de aceptación de 5.4: cualquier persona puede arrancar y probar el proyecto siguiendo solo el README, sin instalar herramientas adicionales (RA-03).
Despliegue	Un solo archivo JAR que se ejecuta con java -jar, junto al XAMPP que ya existe.	sitra-1.0.0.jar sobre Java 21	Contenedores Docker o una nube de pago	Cero presupuesto y cero personal: corre en un equipo que ya existe, sin instalar ni operar otra plataforma (3.5).
Diagramas	PlantUML: herramienta gratuita que dibuja diagramas UML a partir de texto.	Versión libre (los .puml están en docs/diagramas/)	Herramientas gráficas con licencia	Los diagramas son archivos de texto: se guardan en Git junto al código y se puede ver su historia de cambios.
Control de versiones	Git (historial de cambios) y GitHub (publicación en línea).	Git 2.x y repositorio público en GitHub	No se evaluaron alternativas	Es una exigencia del curso; además el historial sirve como evidencia de cómo evolucionaron el documento y el código.

Sección 3. Vistas de arquitectura (modelo C4 y UML)

Una **vista de arquitectura** es un diagrama que muestra el sistema desde un punto de vista concreto, como los planos de una casa: uno muestra la fachada, otro la distribución de las habitaciones y otro las instalaciones eléctricas. Esta sección presenta cinco vistas. Las dos primeras siguen el modelo C4 (Brown, s.f.), que va de lo general a lo particular; las otras tres

usan diagramas UML de clases, de secuencia y de despliegue. Todas se dibujaron con PlantUML, una herramienta gratuita que genera los diagramas a partir de texto, y corresponden al código entregado: los nombres de clases, puertos y tecnologías son los reales. Las fuentes (.puml) y las imágenes en alta resolución están en docs/diagramas/ del repositorio.

Cada vista tiene dos apartados de apoyo: la **leyenda**, que explica qué significa cada forma del dibujo, y la **lectura en 30 segundos**, que cuenta en palabras sencillas lo que muestra el diagrama para quien no esté acostumbrado a leerlos.

3.1 Vista de contexto (C4, nivel 1)

La vista de contexto es la más general: muestra a SITRA como una caja cerrada y responde quién lo usa y con qué otros sistemas se comunica, sin entrar en cómo está hecho por dentro (ver Figura 1).

Figura 1

Vista de contexto: SITRA, sus actores y el sistema externo



Nota. Elaboración propia con PlantUML. La fuente editable está en docs/diagramas/ del repositorio.

Leyenda

- La figura de persona representa un **actor**: una persona o un sistema que inicia una acción. El rectángulo representa un sistema.
- El texto sobre cada flecha dice qué historia de usuario viaja por esa relación y, entre corchetes, la forma de comunicación (el protocolo).

Lectura en 30 segundos

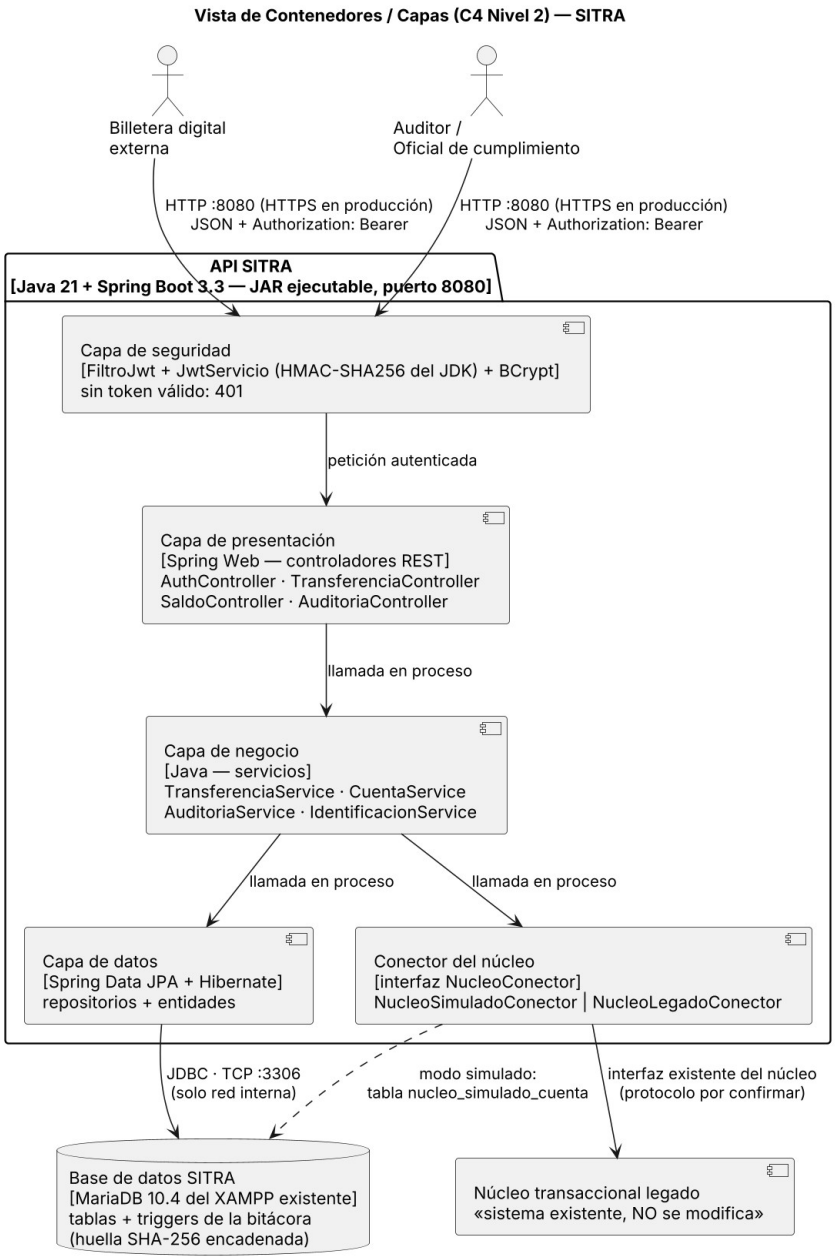
SITRA se ubica en medio, entre la billetera y el núcleo legado. La billetera solo puede consultar saldos y registrar transferencias (HU-01 y HU-02), y el auditor solo puede consultar y verificar la bitácora (HU-03). Los dos se identifican con un token JWT, una credencial digital firmada que presentan en cada solicitud. Nadie de afuera habla directamente con el núcleo: solo SITRA, y usando la interfaz que el núcleo ya tiene. El protocolo exacto de esa interfaz aparece como “por confirmar con el cliente” porque el caso no lo dice; averiguarlo es el primer paso de “trabajar con lo que se tiene”.

3.2 Vista de contenedores y capas (C4, nivel 2)

La vista de contenedores abre la caja de SITRA. En el modelo C4, un **contenedor** es cada programa que se ejecuta por separado o cada lugar donde se guardan datos. La Figura 2 muestra también las capas internas de la aplicación descritas en la Sección 2.1.

Figura 2

Vista de contenedores: la API de SITRA con sus capas y la base de datos



Nota. Elaboración propia con PlantUML. La fuente editable está en docs/diagramas/ del repositorio.

Leyenda

- El marco grande es el contenedor ejecutable (la aplicación); las cajas de su interior son las capas, y el cilindro es la base de datos.
- Entre corchetes aparece la tecnología concreta, la misma de la Tabla 8. Las etiquetas de las flechas indican el protocolo y el puerto; la línea punteada solo se usa en modo simulado.

Lectura en 30 segundos

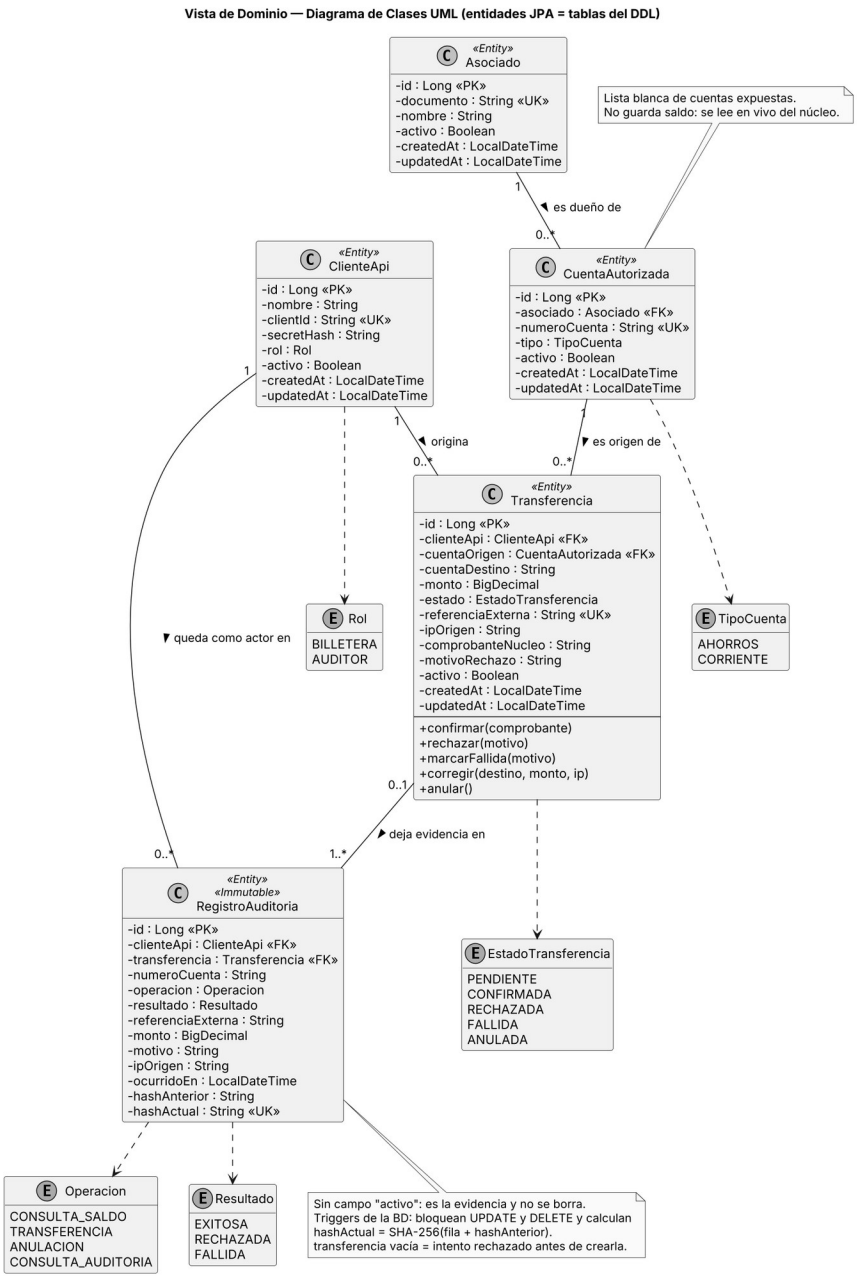
Solo hay dos contenedores: la API de SITRA, que es un único archivo ejecutable de Java 21 con Spring Boot 3.3 y atiende en el puerto 8080, y la base de datos MariaDB 10.4 del XAMPP que ya existe, que atiende en el puerto 3306 solo dentro de la red interna y además contiene los triggers de la bitácora. Dentro de la API, toda petición pasa primero por el filtro JWT; si no trae un token válido, se responde 401 (“no autorizado”) sin llegar a los controladores. Después baja por la presentación y el negocio, y desde el negocio va a los datos o al conector del núcleo. El conector tiene dos versiones, la real y un simulador, y se cambia una por otra con configuración, sin tocar el código.

3.3 Vista de dominio: diagrama de clases UML

El diagrama de clases muestra los conceptos del negocio que maneja el sistema (asociado, cuenta, transferencia, bitácora), qué datos guarda cada uno y cómo se relacionan. Cada clase corresponde a una tabla de la base de datos (ver Figura 3 y Tabla 9).

Figura 3

Diagrama de clases: entidades, atributos con su tipo, llaves y multiplicidades



Nota. Elaboración propia con PlantUML. La fuente editable está en docs/diagramas/ del repositorio.

Leyenda

- «PK» marca la llave primaria (el identificador único de cada registro), «FK» la llave foránea (un dato que apunta a otra tabla) y «UK» un valor que no se puede repetir.
- «Entity» es una clase que JPA guarda como tabla; «Immutable» indica que JPA nunca la modifica después de crearla. La letra E marca una enumeración, es decir, una lista cerrada de valores posibles, como los estados de una transferencia.
- Los números en las líneas son la **multiplicidad**: cuántos objetos de un lado se relacionan con el otro (1 = uno, 0..* = cero o muchos, 0..1 = cero o uno, 1..* = uno o muchos).

Lectura en 30 segundos

Hay cinco entidades. Transferencia es la principal (HU-01): la origina un ClienteApi (la billetera) desde una CuentaAutorizada, que pertenece a un Asociado. Sus métodos son las reglas de estado: confirmar, rechazar, marcar como fallida, corregir y anular. Cada transferencia deja uno o más RegistroAuditoria, y cada registro guarda quién hizo la operación, qué hizo, sobre qué cuenta, con qué resultado, desde qué dirección IP (la dirección de red del equipo que hizo la solicitud) y cuándo.

Hay dos decisiones que conviene destacar. CuentaAutorizada no guarda el saldo, porque el saldo siempre se lee del núcleo, que es la única fuente de verdad; si SITRA guardara una copia, podría quedar desactualizada. Y RegistroAuditoria no tiene el campo “activo” que tienen las demás entidades, porque es la evidencia y no se puede borrar de ninguna forma, ni siquiera de manera lógica.

Tabla 9

Correspondencia entre las clases del programa y las tablas de la base de datos

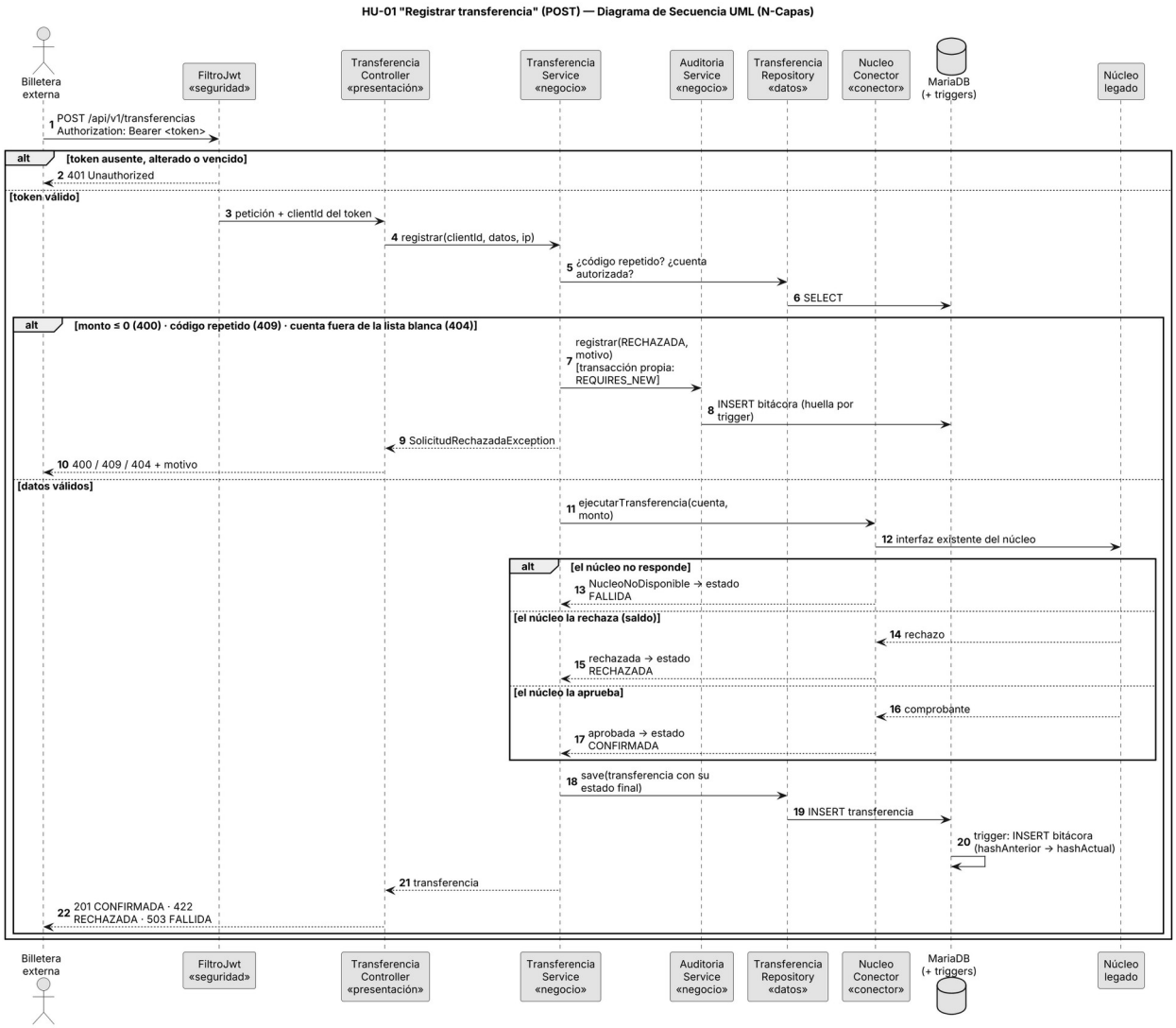
Clase (Java)	Tabla (MariaDB)	Qué representa	Cómo se borra	Dónde aparece en la secuencia (3.4)
Asociado	asociado	La persona asociada a la cooperativa, dueña de una o más cuentas. Solo se guardan su documento y su nombre (datos mínimos, Ley 1581).	Lógico: campo activo = falso.	Es el dueño de la cuenta de origen.
ClienteApi	cliente_api	El sistema o la persona que usa SITRA: la billetera (rol BILLETERA) o el auditor (rol AUDITOR), con su secreto guardado como huella BCrypt.	Lógico: desactivarlo bloquea de inmediato todos sus tokens.	Viene dentro del token (paso 3).
CuentaAutorizada	cuenta_autorizada	Una cuenta de la lista blanca: las únicas que la billetera puede ver y usar. No guarda saldo.	Lógico: campo activo = falso.	Se valida en el servicio (pasos 5 y 6).
Transferencia	transferencia	Una solicitud de mover dinero, con su monto, su destino, su código de operación y su estado (CONFIRMADA, RECHAZADA, FALLIDA o ANULADA).	Lógico: estado ANULADA y activo = falso; la fila nunca desaparece.	Se inserta con su estado final (pasos 18 y 19).
RegistroAuditoria	registro_auditoria	Una fila de la bitácora: quién, qué, sobre qué cuenta, con qué resultado, desde qué IP y cuándo, con su huella encadenada.	Ninguno: un trigger impide modificarla o borrarla.	El rechazo lo escribe el servicio (pasos 7 y 8) y el resto el trigger (paso 20).

3.4 Vista dinámica: diagrama de secuencia UML de la HU-01

El diagrama de secuencia muestra, paso a paso y en orden, cómo colaboran las piezas del sistema cuando la billetera registra una transferencia. Se lee de arriba hacia abajo: cada flecha es un mensaje de una pieza a otra (ver Figura 4).

Figura 4

Secuencia de la HU-01 (POST): camino principal y caminos alternos 401, 400/404/409, 422 y 503



Nota. Elaboración propia con PlantUML. La fuente editable está en docs/diagramas/ del repositorio.

Leyenda

- La flecha continua es una llamada y la flecha punteada es una respuesta. La flecha que vuelve hacia MariaDB representa al trigger, que actúa solo dentro de la base de datos.

- El recuadro “alt” agrupa caminos alternativos que dependen de una condición, como una instrucción “si pasa esto, entonces…”. Los pasos están numerados para poder citarlos.

Lectura en 30 segundos

Sin un token válido, el filtro responde 401 y la petición nunca llega al controlador. Si el monto no es válido, el código de operación ya existe o la cuenta no está en la lista blanca, la transferencia no se crea: el servicio deja el intento como RECHAZADO en la bitácora, en una transacción propia para que no se pierda, y responde 400, 409 o 404 según el caso.

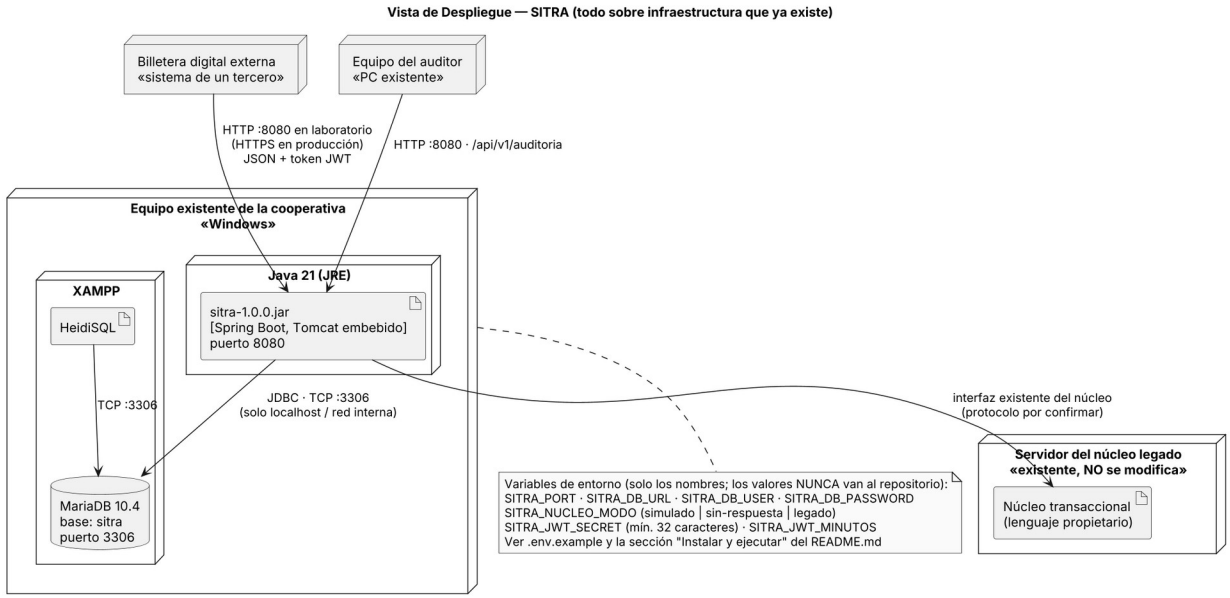
Si los datos son válidos, el servicio llama al núcleo por medio del conector y guarda la transferencia una sola vez, con su estado final: FALLIDA (503) si el núcleo no responde, RECHAZADA (422) si el núcleo la rechaza, por ejemplo por saldo insuficiente, y CONFIRMADA (201) si la aprueba. En los tres casos el trigger de MariaDB escribe la bitácora con su huella, de modo que ningún camino queda sin evidencia. El significado de cada código de respuesta está en la Tabla 14.

3.5 Vista de despliegue

La vista de despliegue muestra en qué equipos físicos se instala y se ejecuta cada parte del sistema y cómo se comunican por la red (ver Figura 5).

Figura 5

Vista de despliegue: todo corre en un equipo que la cooperativa ya tiene



Nota. Elaboración propia con PlantUML. La fuente editable está en docs/diagramas/ del repositorio.

Leyenda

- La caja en tres dimensiones es un **nodo**: una máquina o un entorno de ejecución. La hoja con la esquina doblada es un **artefacto**: lo que se ejecuta en ese nodo. El cilindro es la base de datos.
- El texto de las flechas indica el protocolo y el puerto. La nota lista las variables de entorno sensibles solo por su nombre, nunca con su valor.

Lectura en 30 segundos

No se compra ni se instala ninguna plataforma nueva: en un equipo de la cooperativa ya existen XAMPP (con MariaDB y HeidiSQL) y Java. SITRA es un solo archivo JAR que se ejecuta con el comando java -jar y atiende en el puerto 8080; se conecta a MariaDB por el puerto 3306, que solo se usa dentro del equipo o de la red interna. La billetera y el auditor entran por el

puerto 8080 con su token. El núcleo legado sigue en su propio servidor y solo el conector de SITRA le habla. La Tabla 10 detalla los puertos y las variables de configuración.

Tabla 10

Puertos y variables de entorno del despliegue

Elemento	Valor	Variable donde se configura	Para qué sirve
Puerto de la API	8080. En el laboratorio se usa HTTP; en producción se usará HTTPS, que cifra la comunicación.	SITRA_PORT	Es la “puerta” por la que la billetera y el auditor envían sus solicitudes a SITRA.
Dirección de la base de datos	jdbc:mariadb:// localhost:3306/sitra	SITRA_DB_URL	Le dice a SITRA en qué equipo, en qué puerto y en qué base de datos están sus tablas.
Usuario y contraseña de la base de datos	Aquí solo se escribe el nombre de la variable: el valor nunca se guarda en el repositorio. En producción se usa el usuario sitra_app, de mínimo privilegio.	SITRA_DB_USER y SITRA_DB_PASSWORD	Permiten que SITRA entre a la base sin dejar la contraseña escrita en el código. El usuario sitra_app no puede borrar datos ni quitar los triggers (riesgo R-04).
Modo del conector del núcleo	simulado, sin-respuesta o legado.	SITRA_NUCLEO_MODO	Elige con qué núcleo habla SITRA: el simulador, el simulador que no responde (para probar el 503) o el núcleo real. Es la métrica de AC-03.
Secreto de firma del token	Mínimo 32 caracteres. Si falta, se genera uno aleatorio al arrancar.	SITRA_JWT_SECRET	Es la clave con la que se firman los tokens. Quien la conozca podría fabricar tokens falsos, por eso nunca se versiona (riesgo R-01).
Vida del token	30 minutos por defecto.	SITRA_JWT_MINUTOS	Limita el tiempo durante el cual un token robado podría usarse.

Nota. La correspondencia con el repositorio está en .env.example (nombres de las variables sin valores), en src/main/resources/application.properties (que lee esas variables) y en la sección “Instalar y ejecutar” del README.md.

Sección 4. Registro de decisiones de arquitectura (ADR)

Un **ADR** (registro de decisión de arquitectura) documenta una decisión importante del diseño con una plantilla fija: estatus, fecha, contexto (el problema y las alternativas), la decisión tomada y sus consecuencias, es decir, lo que se gana y lo que se sacrifica (Nygard, 2011). Los ADR no se editan: si una decisión cambia, se escribe uno nuevo que la reemplaza. Así se hizo con la base de datos en el ADR-002 y con la identificación provisional en el ADR-005.

SITRA tiene cinco decisiones críticas, cada una guardada en docs/adr/ del repositorio. La Tabla 11 las resume y a continuación se transcribe cada una desde esos mismos archivos, precedida de un resumen en palabras sencillas.

Tabla 11

Resumen de las decisiones de arquitectura

ADR	Decisión	Atributo o restricción que la obliga	Qué se sacrifica
ADR-001	Usar el patrón N-Capas, con un conector intercambiable para el núcleo legado.	AC-03 Mantenibilidad; no existe ambiente de pruebas del núcleo; cero personal técnico.	Se escribe más código que en un MVC simple, y la lógica de negocio no queda totalmente aislada del framework Spring.
ADR-002	Usar la base de datos MariaDB del XAMPP que ya existe, en lugar de PostgreSQL.	AC-01 integridad de la bitácora; RA-02 cero presupuesto; RA-03 cero personal.	Se pierden algunas funciones propias de PostgreSQL y se usa una versión de MariaDB (10.4) que ya no recibe actualizaciones.
ADR-003	La trazabilidad la garantiza la base de datos con triggers, y los intentos rechazados siempre se guardan.	AC-01 Seguridad (responsabilidad y no repudio); exigencia del auditor.	Parte de la lógica vive en la base de datos, donde es menos visible que en el código Java, y cada operación hace una escritura adicional.
ADR-004	Actualizar (PUT) una transferencia significa corregirla y reintentarla solo si no movió dinero; anular pide el reverso al núcleo.	AC-01 (la evidencia debe coincidir con lo ocurrido) y AC-02 (reintentar sin duplicar).	El PUT no es genérico: quien lo usa debe conocer la regla, que está en el README y la prueba la colección de Postman.

ADR	Decisión	Atributo o restricción que la obliga	Qué se sacrifica
ADR-005	Token JWT firmado con HMAC del propio Java y secretos guardados con BCrypt, sin instalar Spring Security completo.	AC-01 (401 sin token; responsable en la bitácora); RA-02; RA-03.	Hay unas 240 líneas de código de seguridad propio que cuidar, y no existe renovación automática de tokens.

ADR-001: Patrón N-Capas con un conector intercambiable para el núcleo legado

En palabras sencillas

Se decidió organizar SITRA por capas, como un edificio de pisos donde cada piso tiene una sola función, y conectar el núcleo por medio de una sola “pieza enchufable”. Esa pieza tiene dos versiones: una de práctica (el simulador) y la real. Así se puede probar todo sin tocar el núcleo y, cuando se conozca cómo conectarse a él, solo se cambia esa pieza.

Estatus

Aceptada

Fecha

24 de octubre de 2026

Contexto

SITRA debe exponer consulta de saldo y transferencias de un núcleo legado de 20 años, sin documentación, que no se puede modificar y del que no existe ambiente de pruebas. La cooperativa no tiene personal técnico: quien herede el sistema debe entenderlo sin ayuda.

Atributos en juego: **AC-03 Mantenibilidad** (modularidad y capacidad de ser probado) y **AC-02 Fiabilidad**. Restricciones: núcleo intocable, sin ambiente de pruebas, cero personal técnico, cero presupuesto.

Alternativas consideradas:

- **MVC monolítico:** el controlador hablaría directo con el núcleo y la base de datos.

- **Hexagonal (puertos y adaptadores):** máxima independencia, pero más clases, interfaces y conceptos.
- **Microservicios:** varios servicios que operar y monitorear.
- **N-Capas:** seguridad → presentación → negocio → datos.

Decisión

Decidimos usar N-Capas (seguridad, presentación, negocio y datos) y que el núcleo se use solo a través de la interfaz `NucleoConector`, con dos implementaciones que se cambian por configuración (`sitra.nucleo.modo`): `NucleoSimuladoConector` para pruebas y `NucleoLegadoConector` para producción, porque AC-03 exige poder probar todo sin el núcleo real y la restricción de cero personal técnico exige la estructura más estándar posible.

Se descarta MVC porque no deja dónde poner el simulador sin tocar código de negocio. Se descarta Hexagonal completo porque su costo en clases y curva de aprendizaje contradice la restricción de cero personal; de él solo se toma la interfaz del conector. Se descarta Microservicios porque exige personal y presupuesto que no existen.

Consecuencias y compromisos (trade-offs)

- **Positivas:** cualquier desarrollador Java reconoce la estructura (controller, service, repository, model); el servicio no sabe si habla con el núcleo real o con el simulador; cambiar la integración toca una sola clase y un valor de configuración.
- **Negativas:** más código que un MVC simple (servicios y repositorios separados). El dominio no queda totalmente aislado de Spring, como sí lo haría en Hexagonal.
- **Deuda técnica asumida:** `NucleoLegadoConector` responde “no disponible” porque el protocolo del núcleo está por confirmar con el cliente. Condición para revisarla: cuando

el cliente confirme cómo expone sus datos el núcleo (archivo, base de datos o servicio), se implementa solo esa clase, con una espera máxima de 5 s (AC-02).

ADR-002: MariaDB (la del XAMPP existente) en lugar de PostgreSQL

En palabras sencillas

Al principio se eligió la base de datos PostgreSQL, pero los equipos de la cooperativa ya tienen instalada MariaDB dentro del paquete XAMPP. Como no hay dinero ni personal para instalar y mantener otra, se usa la que ya existe, que cumple todo lo necesario para proteger la bitácora.

Estatus

Aceptada — reemplaza la elección de PostgreSQL 16 de la Sección 2.2 del DAS v0.4.

Fecha

24 de octubre de 2026

Contexto

En el DAS v0.4 se eligió PostgreSQL 16. En la sustentación de la Clase 4 el modelo se probó en vivo en los equipos disponibles, que tienen XAMPP con MariaDB y HeidiSQL ya instalados. La restricción RA-02 dice que solo se trabaja con lo que se tiene y la RA-03 que no hay personal técnico para instalar y mantener otro motor.

Atributos en juego: **AC-01 Seguridad** (integridad de la bitácora), **AC-03 Mantenibilidad**. Restricciones: cero presupuesto, cero personal técnico, usar la infraestructura existente.

Alternativas consideradas:

- **PostgreSQL 16:** muy completo, pero hay que instalarlo y administrarlo aparte.
- **MariaDB 10.4+ del XAMPP:** ya instalado, gratuito, con HeidiSQL para operarlo.

- **H2 en memoria:** no sirve como evidencia porque se borra al apagar.

Decisión

Decidimos usar MariaDB 10.4+ incluida en XAMPP, porque cumple todo lo que exige AC-01 —llaves foráneas, CHECK (MariaDB los aplica desde la 10.2), UNIQUE, transacciones InnoDB y triggers que bloquean la bitácora— y porque ya está instalada, cumpliendo RA-02 y RA-03.

Se descarta PostgreSQL porque, aunque es técnicamente equivalente para este caso, exige instalar y mantener un motor nuevo sin personal que lo haga. Se descarta H2 porque no conserva la evidencia.

Consecuencias y compromisos (trade-offs)

- **Positivas:** cero instalación; el mismo motor sirve para desarrollo, pruebas y la demostración en clase; HeidiSQL permite al auditor consultar la bitácora sin programar.
- **Negativas:** se pierden funciones propias de PostgreSQL (índices parciales, pgcrypto). Se reemplazan: el índice parcial por uno normal y la huella por `SHA2()` de MariaDB. Los scripts PostgreSQL de la Clase 4 quedan en el historial de Git, no en la versión actual.
- **Deuda técnica asumida:** XAMPP trae MariaDB 10.4, una versión sin soporte del fabricante. Condición para revisarla: si el sistema pasa a producción real, actualizar a una versión con soporte (10.11 LTS); los scripts no cambian.

ADR-003: La trazabilidad la garantiza la base de datos, no solo el código

En palabras sencillas

La bitácora no depende de que el programa “se acuerde” de escribirla: la propia base de datos la escribe sola con reglas automáticas (triggers), no deja modificarla ni borrarla y le pone a

cada registro una huella digital encadenada con la del anterior. Si alguien alterara un registro, la cadena se rompería y se notaría. Los intentos rechazados también se guardan.

Estatus

Aceptada

Fecha

24 de octubre de 2026

Contexto

La trazabilidad es el atributo crítico del caso (**AC-01 Seguridad**: responsabilidad y no repudio): el auditor la exige y de ella depende la licencia de operación. En la Clase 4 se comprobó que, si la bitácora la escribe solo la aplicación, una transferencia insertada directamente por SQL no deja rastro. Además el auditor pidió registrar también los intentos rechazados.

Alternativas consideradas:

- **Solo la aplicación escribe la bitácora:** simple, pero cualquier acceso directo a la base de datos (HeidiSQL, otro programa) se salta el registro.
- **Servicio de auditoría en la nube:** con costo recurrente (prohibido por RA-02).
- **Triggers en la base de datos + registro de rechazos desde el servicio.**

Decisión

Decidimos que la base de datos garantice la bitácora con triggers: toda transferencia que se inserte o cambie deja su fila automáticamente; la bitácora no se puede modificar ni borrar; y cada fila guarda una huella SHA-256 encadenada con la anterior. Los intentos rechazados se guardan así: si la base de datos o la validación impiden crear la transferencia, el servicio escribe la fila RECHAZADA en la bitácora en una transacción propia (`REQUIRES_NEW`); si el núcleo la

rechaza (saldo insuficiente), la transferencia se guarda con estado RECHAZADA. Todo con funciones gratuitas de MariaDB.

Consecuencias y compromisos (trade-offs)

- **Positivas:** ninguna transferencia queda sin evidencia, sin importar por dónde entre; los rechazados nunca se pierden; la regla no depende de que un programador "se acuerde".
- **Negativas:** parte de la lógica vive en la base de datos (triggers y un procedimiento), que es menos visible que el código Java. Cada inserción hace una escritura extra en la bitácora.
- **Deuda técnica asumida:** el procedimiento `registrar_transferencia` repite en SQL la regla del `TransferenciaService`, para poder operar y demostrar desde HeidiSQL. Condición para revisarla: si las dos versiones empiezan a diferir, se deja una sola y la otra la llama.

ADR-004: En el CRUD de Transferencia, actualizar (PUT) es "corregir y reintentar"

En palabras sencillas

Una transferencia que ya movió dinero no se puede "editar", porque la evidencia dejaría de coincidir con lo que pasó. Solo se pueden corregir y reintentar las que no movieron dinero (rechazadas o fallidas), y anular una transferencia exige primero devolver el dinero en el núcleo.

Estatus

Aceptada

Fecha

6 de noviembre de 2026

Contexto

La plantilla exige un CRUD completo de la entidad principal de la HU-01 (Transferencia). Pero una transferencia no es un dato cualquiera: si ya movió dinero en el núcleo, cambiar su monto o su destino en SITRA dejaría la evidencia diciendo algo distinto de lo que pasó. Además, en la Clase 5 se identificó un hueco de fiabilidad: si el núcleo no respondía (FALLIDA, 503), la billetera no podía reintentar con el mismo código de operación, porque el código ya existía (409).

Atributos en juego: **AC-01 Seguridad** (la evidencia debe coincidir con lo ocurrido) y **AC-02 Fiabilidad** (poder recuperarse de un fallo del núcleo sin duplicar operaciones).

Alternativas consideradas:

- **PUT libre sobre cualquier campo:** cumple la plantilla, pero permite "maquillar" una transferencia confirmada.
- **No tener PUT:** incumple el CRUD completo.
- **PUT solo sobre lo que no movió dinero (RECHAZADA o FALLIDA):** corregir destino o monto y volver a enviarla al núcleo con el mismo código de operación.

Decisión

Decidimos que `PUT /api/v1/transferencias/{id}` solo acepte transferencias RECHAZADAS o FALLIDAS: corrige `cuentaDestino` y `monto`, conserva el código de operación y vuelve a pasar por el núcleo (200 si queda CONFIRMADA, 422 si el núcleo la vuelve a rechazar, 503 si sigue sin responder). Sobre una CONFIRMADA responde 409. Y `DELETE` es un borrado lógico (ANULADA, `activo = false`, 204): si la transferencia ya había movido dinero, primero se pide el reverso al núcleo; si el núcleo no responde, no se anula nada (503).

Consecuencias y compromisos (trade-offs)

- **Positivas:** el UPDATE tiene sentido de negocio; resuelve el reintento después de un 503 sin romper la idempotencia; la bitácora (trigger de UPDATE) guarda el intento original y el reintento, así que nada se borra ni se maquilla.
- **Negativas:** el PUT no es "genérico" como en el ejemplo de la plantilla; quien lo use debe conocer la regla (está en el README y la colección Postman la prueba: 409 y 200).
- **Deuda técnica asumida:** el reverso depende de que el núcleo real ofrezca una operación para devolver dinero; hoy solo existe en el simulador. Condición para revisarla: al implementar `NucleoLegadoConector` (ADR-001), confirmar con el cliente cómo se reversa una operación.

ADR-005: Token JWT firmado con HMAC del JDK y secretos con BCrypt, sin Spring Security completo

En palabras sencillas

Cada solicitud debe demostrar quién la hace. Para eso, la billetera y el auditor reciben un token, una especie de pase firmado que vence a los 30 minutos y no se puede falsificar. Las claves de los clientes se guardan transformadas en una huella irreversible, nunca como texto legible. Todo se hizo con herramientas que ya trae Java, sin instalar plataformas nuevas.

Estatus

Aceptada — reemplaza la identificación provisional por encabezado `X-Cliente-Id` del arranque (Clase 5).

Fecha

6 de noviembre de 2026

Contexto

El consumidor de SITRA es otro sistema (la billetera), no una persona con navegador. Cada petición debe probar quién la hace, porque ese dato queda en la bitácora como responsable (**AC-01**: responsabilidad y no repudio; meta: 401 en el 100 % de los intentos sin token). En el arranque se usó el encabezado `X-Cliente-Id`, que cualquiera puede falsificar.

Restricciones: cero presupuesto (RA-02) y cero personal técnico (RA-03): no hay quién instale ni opere un servidor de identidad.

Alternativas consideradas:

- **Sesiones en servidor:** pensadas para navegadores; no aplican a un sistema que llama a otro.
- **Servidor de identidad (por ejemplo Keycloak, OAuth2):** muy completo, pero es otra plataforma que instalar, actualizar y operar.
- **Spring Security completo + librería JWT externa:** estándar, pero agrega una cadena de filtros y configuración que un mantenedor sin experiencia difícilmente entiende.
- **Token JWT HS256 firmado con `javax.crypto` (incluido en Java) + BCrypt de `spring-security-crypto` + un filtro propio.**

Decisión

Decidimos usar JWT HS256 firmado con HMAC-SHA256 del propio JDK (`JwtServicio`), validado en un solo filtro (`FiltroJwt`) antes de llegar a los controladores, y guardar los secretos de los clientes solo como hash BCrypt (`spring-security-crypto`, sin el resto de Spring Security), porque AC-01 exige identificar a quien llama en cada petición y RA-02/RA-03 exigen no agregar plataformas ni productos. El secreto de firma llega por la variable

`SITRA_JWT_SECRET` (mínimo 32 caracteres); si no existe, se genera uno aleatorio al arrancar. El token vence a los 30 minutos.

Consecuencias y compromisos (trade-offs)

- **Positivas:** 401 sin token en todas las rutas `/api/**` (salvo `/api/v1/auth/token`); el rol viaja firmado y no se puede cambiar sin invalidar la firma (probado en `JwtServicioTest`); los secretos nunca se guardan en texto plano; no hay nada nuevo que instalar.
- **Negativas:** código de seguridad propio (unas 240 líneas en el paquete `security`) que hay que cuidar: comparación de firmas en tiempo constante, se rechaza cualquier algoritmo distinto de HS256 y se valida el vencimiento. No hay renovación de tokens (refresh) ni lista de revocación; se mitiga porque en cada petición se verifica que el cliente siga activo en la base de datos: desactivarlo bloquea sus tokens de inmediato.
- **Deuda técnica asumida:** HTTPS. En el laboratorio la API corre en HTTP (puerto 8080) y el token viaja sin cifrar. Condición para revisarla: antes de producción se activa HTTPS en Spring Boot con un certificado gratuito y se rota `SITRA_JWT_SECRET`.

Sección 5. Entregable de programación ejecutable (corte vertical)

Un **corte vertical** es una funcionalidad completa que atraviesa todas las capas del sistema, como una tajada de un pastel de varios pisos: demuestra que todas las capas encajan y funcionan juntas. El de SITRA implementa el CRUD completo de Transferencia, la entidad principal de la HU-01. Cada solicitud recorre este camino: API REST, filtro JWT, controlador, servicio, repositorio y ORM, base de datos MariaDB con sus triggers, y de regreso. Además se exponen la consulta de saldo (HU-02) y la consulta y verificación de la bitácora (HU-03).

5.1 Repositorio Git

Todo el proyecto está en el repositorio público https://github.com/JorgeMario-V/sitra. La Figura 6 muestra cómo están organizadas sus carpetas.

Figura 6

Estructura de carpetas del repositorio

```
sitra/
├── docs/            DAS.docx, DAS.pdf, adr/ (ADR-001 a ADR-005), diagramas/ (.puml + .png)
├── db/
│   ├── migrations/  V1 tablas · V2 triggers de bitácora · V3 rechazados · V4 IP de origen
│   ├── datos_prueba.sql        clientes (secretos BCrypt), asociados, cuentas y saldos
│   ├── instalar_todo.sql       todo lo anterior en un solo archivo
│   ├── pruebas/                demo_sustentacion.sql, concurrencia.cmd y concurrencia.sh
│   └── produccion/usuario_minimo_privilegio.sql
├── src/main/java/co/coop/sitra/
│   ├── security/     FiltroJwt, JwtServicio, ConfiguracionSeguridad
│   ├── controller/   Auth, Transferencia, Saldo y Auditoria Controller
│   ├── service/      Transferencia, Cuenta, Auditoria e Identificacion Service
│   ├── repository/   repositorios Spring Data JPA
│   ├── model/        entidades JPA (= tablas) y enumeraciones
│   ├── integration/  NucleoConector + simulador + conector legado
│   ├── dto/          cuerpos JSON de entrada y salida
│   └── exception/    ManejadorErrores: errores → códigos HTTP
├── src/main/resources/application.properties   lee las variables de entorno
├── src/test/java/co/coop/sitra/   pruebas unitarias y de integración (5.4)
├── postman/coleccion-corte-vertical.json        19 peticiones con 30 pruebas
├── scripts/generar_evidencias.cmd               genera las salidas de evidencia
├── evidencias/      una carpeta por entrega
├── mvnw, mvnw.cmd, .mvn/   Maven Wrapper (no hay que instalar Maven)
└── .env.example · .gitattributes · .gitignore · pom.xml · README.md
```

Nota. Elaboración propia. Los archivos .java de cada paquete se listan completos en el repositorio.

Política de commits y de ramas

Cada **commit** (cada cambio guardado en el historial) es atómico, es decir, contiene un solo cambio con sentido propio, y su mensaje sigue el formato Conventional Commits: un prefijo que dice el tipo de cambio (feat: funcionalidad nueva, fix: corrección, docs: documentación, test: pruebas, chore: mantenimiento) seguido de una descripción. El archivo .gitignore excluye .env, target/ y los archivos del editor, de modo que el archivo .env con valores reales nunca entra al historial; el archivo .gitattributes fija el tipo de fin de línea de los scripts para que funcionen igual en Windows y en Linux.

El uso de ramas se adoptó a partir de la Clase 5. Las entregas de las Clases 1 a 4 se hicieron directamente sobre la rama principal (main), con commits del 26 de septiembre y del 16 de octubre de 2026. Desde la Clase 5, el trabajo de cada clase se hizo en una rama propia con el prefijo feature/ que luego se unió a main con un commit de unión (merge), de modo que main siempre quedara ejecutable; en el historial se ven las cuatro uniones de las Clases 5 a 8. Los commits de las Clases 5 a 8 tienen fecha del 6 de noviembre porque ese día se subieron al repositorio en línea; antes se trabajaba en el equipo local.

El historial no se reescribe: no se usan comandos que lo modifiquen, como rebase o push forzado, para que las fechas y el orden de los cambios sigan siendo evidencia de cómo avanzó el proyecto. Desde la versión 1.1, las correcciones se hacen en la rama feature/correcciones-trazabilidad, con un commit por hallazgo de la revisión (Anexo B), y cada sesión de trabajo termina con un push, es decir, con los cambios subidos a GitHub.

5.2 Script SQL de la base de datos (migraciones en MariaDB)

La estructura de la base de datos se crea con cuatro scripts SQL que se ejecutan en orden; a cada uno se le llama **migración** porque lleva la base de una versión a la siguiente. V1 crea las tablas; V2 crea los triggers que protegen la bitácora; V3 crea el procedimiento que registra también los intentos rechazados, y V4 es el ALTER (la instrucción que modifica una tabla existente) que agrega la IP de origen a la transferencia, cambio pedido en la Clase 4. El archivo db/instalar_todo.sql los ejecuta todos de una vez junto con los datos de prueba. La Figura 7 muestra V1 completo.

Figura 7

Script V1: tablas, llaves, restricciones CHECK y UNIQUE, e índices

```
-- ========================================================================
-- SITRA — V1: tablas, llaves, CHECK, UNIQUE e índices
-- Motor: MariaDB 10.4+ (XAMPP). Ejecutar en orden: V1, V2, V3, V4.
-- ========================================================================

-- Quién usa SITRA: la billetera externa y el auditor
CREATE TABLE cliente_api (
    id           BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    nombre       VARCHAR(120)  NOT NULL,
    client_id    VARCHAR(80)   NOT NULL UNIQUE,
    secret_hash  VARCHAR(255)  NOT NULL,                          -- BCrypt, nunca el secreto
    rol          VARCHAR(20)   NOT NULL DEFAULT 'BILLETERA',
    activo       BOOLEAN       NOT NULL DEFAULT TRUE,             -- borrado lógico
    created_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at   DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_cliente_rol CHECK (rol IN ('BILLETERA', 'AUDITOR'))
) ENGINE = InnoDB;

-- Asociado de la cooperativa
CREATE TABLE asociado (
    id          BIGINT        NOT NULL AUTO_INCREMENT PRIMARY KEY,
    documento   VARCHAR(30)   NOT NULL UNIQUE,
    nombre      VARCHAR(120)  NOT NULL,
    activo      BOOLEAN       NOT NULL DEFAULT TRUE,              -- borrado lógico
    created_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;

-- Cuenta (lista blanca de cuentas expuestas). NO guarda saldo: el saldo vive en el núcleo.
CREATE TABLE cuenta_autorizada (
    id             BIGINT       NOT NULL AUTO_INCREMENT PRIMARY KEY,
    asociado_id    BIGINT       NOT NULL,
    numero_cuenta  VARCHAR(30)  NOT NULL UNIQUE,
    tipo           VARCHAR(20)  NOT NULL,
    activo         BOOLEAN      NOT NULL DEFAULT TRUE,             -- borrado lógico
    created_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at     DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_cuenta_asociado FOREIGN KEY (asociado_id) REFERENCES asociado(id),
    CONSTRAINT ck_cuenta_tipo CHECK (tipo IN ('AHORROS', 'CORRIENTE'))
) ENGINE = InnoDB;

-- Transacción (transferencia)
CREATE TABLE transferencia (
    id                 BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id     BIGINT         NOT NULL,
    cuenta_origen_id   BIGINT         NOT NULL,
    cuenta_destino     VARCHAR(30)    NOT NULL,
    monto              DECIMAL(14,2)  NOT NULL,
    estado             VARCHAR(20)    NOT NULL DEFAULT 'PENDIENTE',
    referencia_externa VARCHAR(80)    NOT NULL,                  -- código de operación
    comprobante_nucleo VARCHAR(80)    NULL,
    motivo_rechazo     VARCHAR(255)   NULL,                      -- por qué se rechazó
    activo             BOOLEAN        NOT NULL DEFAULT TRUE,      -- borrado lógico
    created_at         DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at         DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_transf_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
    CONSTRAINT fk_transf_cuenta  FOREIGN KEY (cuenta_origen_id) REFERENCES cuenta_autorizada(id),
    CONSTRAINT uq_transf_idempotencia UNIQUE (referencia_externa),    -- llave de idempotencia
    CONSTRAINT ck_transf_monto  CHECK (monto > 0),                    -- CHECK de monto > 0
    CONSTRAINT ck_transf_estado CHECK (estado IN ('PENDIENTE','CONFIRMADA','RECHAZADA','FALLIDA','ANULADA'))
) ENGINE = InnoDB;

-- Bitácora de auditoría (obligatoria: la trazabilidad es el atributo crítico).
-- Guarda también los intentos RECHAZADOS: código, monto intentado y motivo.
-- monto NO tiene CHECK: debe poder guardar lo que se intentó, aunque sea inválido (ej. -50.000).
CREATE TABLE registro_auditoria (
    id                 BIGINT         NOT NULL AUTO_INCREMENT PRIMARY KEY,
    cliente_api_id     BIGINT         NOT NULL,                  -- quién
    transferencia_id   BIGINT         NULL,                      -- vacío si la transferencia no se pudo crear
    numero_cuenta      VARCHAR(30)    NOT NULL,                  -- sobre qué cuenta
    operacion          VARCHAR(30)    NOT NULL,                  -- qué
    resultado          VARCHAR(20)    NOT NULL,                  -- EXITOSA / RECHAZADA / FALLIDA
    referencia_externa VARCHAR(80)    NULL,                      -- código de operación intentado
    monto              DECIMAL(14,2)  NULL,                      -- monto intentado
    motivo             VARCHAR(255)   NULL,                      -- por qué se rechazó
    ip_origen          VARCHAR(45)    NOT NULL DEFAULT 'local',
    ocurrido_en        DATETIME       NOT NULL DEFAULT CURRENT_TIMESTAMP,  -- cuándo
    hash_anterior      CHAR(64)       NULL,
    hash_actual        CHAR(64)       NULL UNIQUE,
    CONSTRAINT fk_audit_cliente FOREIGN KEY (cliente_api_id)   REFERENCES cliente_api(id),
```

```
    CONSTRAINT fk_audit_transf  FOREIGN KEY (transferencia_id) REFERENCES transferencia(id),
    CONSTRAINT ck_audit_operacion CHECK (operacion IN
('CONSULTA_SALDO','TRANSFERENCIA','ANULACION','CONSULTA_AUDITORIA')),
    CONSTRAINT ck_audit_resultado CHECK (resultado IN ('EXITOSA','RECHAZADA','FALLIDA'))
) ENGINE = InnoDB;

-- Simulador del núcleo legado: aquí vive el SALDO (SITRA no guarda saldos)
CREATE TABLE nucleo_simulado_cuenta (
    numero_cuenta  VARCHAR(30)    NOT NULL PRIMARY KEY,
    saldo          DECIMAL(14,2)  NOT NULL,
    CONSTRAINT ck_nucleo_saldo CHECK (saldo >= 0)
) ENGINE = InnoDB;

-- Índices justificados
CREATE INDEX idx_transferencia_cuenta   ON transferencia (cuenta_origen_id);
CREATE INDEX idx_auditoria_cuenta_fecha ON registro_auditoria (numero_cuenta, ocurrido_en);
```

Nota. Archivo db/migrations/V1__esquema_inicial.sql del repositorio, transcrito sin cambios.

La Figura 8 muestra el inicio de V4, el cambio pedido en la Clase 4. El resto del archivo vuelve a crear los triggers para que la bitácora también copie la IP.

Figura 8

Extracto del script V4: la transferencia guarda la IP de origen

```
-- 1) La transacción guarda desde qué IP se originó
ALTER TABLE transferencia
    ADD COLUMN ip_origen VARCHAR(45) NULL AFTER referencia_externa;
```

Nota. Extracto de db/migrations/V4__ip_origen_en_transferencia.sql.

La Tabla 12 muestra dónde se cumple cada requisito evaluable de la plantilla dentro de estos scripts.

Tabla 12

Cumplimiento de los requisitos evaluables del script SQL

Requisito de la plantilla	Qué significa	Dónde se cumple en SITRA
Llave primaria (PK) en toda tabla	Cada fila tiene un identificador único que no se repite y nunca está vacío.	Columna id BIGINT AUTO_INCREMENT PRIMARY KEY en cinco tablas; nucleo_simulado_cuenta usa numero_cuenta como llave.
Llave foránea (FK) con REFERENCES	Un dato solo puede apuntar a un registro que exista en otra tabla; así no quedan transferencias de cuentas inexistentes.	cuenta_autorizada apunta a asociado; transferencia apunta a cliente_api y a cuenta_autorizada; registro_auditoria apunta a cliente_api y a transferencia.

Requisito de la plantilla	Qué significa	Dónde se cumple en SITRA
Al menos una restricción CHECK	La base de datos rechaza valores que no cumplan una condición, aunque el programa se equivoque.	monto > 0 (ck_transf_monto) y las listas cerradas de estado, rol, tipo de cuenta, operación y resultado; saldo ≥ 0 en el simulador.
Al menos una restricción UNIQUE	Un valor no se puede repetir en toda la tabla.	referencia_externa es la llave de idempotencia (uq_transf_idempotencia): impide registrar dos veces la misma transferencia. También son únicos client_id, documento, numero_cuenta y hash_actual.
Borrado lógico	En lugar de eliminar la fila, se marca como inactiva y se conserva.	Campo activo en asociado, cliente_api, cuenta_autorizada y transferencia; el DELETE de la API deja la transferencia ANULADA y con activo = falso. La bitácora no lo tiene a propósito: es evidencia y no se borra.
Índices justificados	Estructuras que aceleran las búsquedas frecuentes, como el índice alfabético de un libro.	idx_transferencia_cuenta acelera el historial por cuenta; idx_auditoria_cuenta_fecha acelera la consulta del auditor por cuenta y fechas (HU-03, meta de 5 segundos).

Verificación ejecutada en MariaDB

Para demostrar que las reglas funcionan dentro de la base de datos, y no solo en el programa, se ejecutan db/instalar_todo.sql y db/pruebas/demo_sustentacion.sql. La Figura 9 resume lo que responde MariaDB en cada paso.

Figura 9

Resumen de la verificación de las reglas de la base de datos

```
Paso 1  transferencia 300.000 por SQL -> bitácora: TRANSFERENCIA · EXITOSA (la escribió el trigger)
Paso 2  mismo código                  -> ERROR 1062 Duplicate entry ... 'uq_transf_idempotencia'
        monto -50.000                 -> ERROR 4025 CONSTRAINT 'ck_transf_monto' failed
Paso 4  CALL con código repetido      -> RECHAZADA · queda en la bitácora con su motivo
        CALL con -50.000              -> RECHAZADA · queda en la bitácora con su motivo
        CALL con 5.000.000            -> RECHAZADA · transferencia RECHAZADA + bitácora
Paso 5  verificación de la cadena     -> cadena OK · huella OK en todas las filas
        UPDATE / DELETE en bitácora   -> ERROR 1644: no se puede modificar / borrar
Usuario de mínimo privilegio: DROP TRIGGER -> ERROR 1142 · DELETE -> ERROR 1142
Alteración simulada (sin triggers)    -> fila modificada: ALTERADO · fila borrada: la siguiente ROTA
```

Nota. Las salidas completas están en evidencias/entrega-07-pruebas-y-tradeoffs/: resultado_verificacion_mariadb.txt (pasos 1 a 5) y resultado_minimo_privilegio.txt (usuario sin permisos de borrar ni de quitar triggers); las dos las genera scripts\generar_evidencias.cmd. El

archivo resultado_verificacion_ddl.txt de la entrega 4 es histórico: corresponde al esquema en PostgreSQL anterior al ADR-002.

Los códigos de error de la Figura 9 son la prueba de que la base de datos se defiende sola: 1062 significa valor repetido, 4025 que no se cumplió una restricción CHECK, 1644 que un trigger detuvo la operación y 1142 que el usuario no tiene permiso para hacerla.

5.3 Código del backend por capas (Spring Boot y JPA)

El **backend** es la parte del sistema que no se ve: recibe las solicitudes, aplica las reglas y guarda los datos. Lo evaluable en esta sección es la separación de capas: cada petición entra por el filtro JWT; el controlador solo traduce a HTTP; el servicio aplica las reglas y es el único que conoce el conector del núcleo, y el repositorio habla con MariaDB. La Tabla 13 presenta los endpoints, es decir, las direcciones de la API y lo que hace cada una.

Tabla 13

Endpoints de la API de SITRA

Método y ruta	Qué hace (en palabras sencillas)	Quién puede usarla	Respuestas posibles
POST /api/v1/auth/token	Cambia el identificador y el secreto del cliente (comparado con BCrypt) por un token JWT que vale 30 minutos.	Cualquier cliente registrado y activo	200 · 401
POST /api/v1/transferencias	Crear (HU-01): valida los datos, pide la operación al núcleo y guarda la transferencia con su estado final.	BILLETERA	201 · 400 · 401 · 403 · 404 · 409 · 422 · 503
GET /api/v1/transferencias	Consultar: lista las transferencias activas del cliente que pregunta.	BILLETERA	200 · 401 · 403
GET /api/v1/transferencias/{id}	Consultar: muestra una transferencia activa propia.	BILLETERA	200 · 401 · 403 · 404
PUT /api/v1/transferencias/{id}	Actualizar (ADR-004): corrige y reintenta una transferencia RECHAZADA o FALLIDA.	BILLETERA	200 · 400 · 401 · 403 · 404 · 409 · 422 · 503

Método y ruta	Qué hace (en palabras sencillas)	Quién puede usarla	Respuestas posibles
DELETE /api/v1/transferencias/{id}	Eliminar de forma lógica: si la transferencia movió dinero, primero pide el reverso al núcleo; después la marca ANULADA y con activo = falso.	BILLETERA	204 · 401 · 403 · 404 · 409 · 503
GET /api/v1/cuentas/{numero}/saldo	HU-02: devuelve el saldo leído en ese momento del núcleo y deja la consulta en la bitácora.	BILLETERA	200 · 401 · 403 · 404 · 503
GET /api/v1/auditoria?cuenta=&desde=&hasta=	HU-03: devuelve la bitácora de una cuenta en un rango de fechas.	AUDITOR	200 · 400 · 401 · 403
GET /api/v1/auditoria/verificacion	HU-03: recalcula toda la cadena de huellas; responde íntegra = verdadero si nadie alteró ni borró nada.	AUDITOR	200 · 401 · 403

Nota. Cuando un cliente usa una ruta que no corresponde a su rol, SITRA responde 403 y deja el intento en la bitácora. GET, POST, PUT y DELETE son las operaciones de HTTP para consultar, crear, modificar y eliminar.

Cada respuesta de la API lleva un **código de estado HTTP**, un número de tres cifras definido por el estándar HTTP (Fielding et al., 2022) que indica si la solicitud funcionó y, si no, por qué. Los que empiezan por 2 indican éxito, los que empiezan por 4 un problema de la solicitud y los que empiezan por 5 un problema del servidor o de un sistema del que depende. La Tabla 14 explica los que usa SITRA.

Tabla 14

Códigos de estado HTTP que responde SITRA

Código	Nombre en el estándar	Qué significa en palabras sencillas	Cuándo lo responde SITRA
200	OK	La solicitud funcionó y la respuesta trae los datos pedidos.	Consultas, token entregado y corrección (PUT) que quedó confirmada.

Código	Nombre en el estándar	Qué significa en palabras sencillas	Cuándo lo responde SITRA
201	Created (creado)	Se creó un registro nuevo.	La transferencia quedó CONFIRMADA por el núcleo.
204	No Content (sin contenido)	La acción se hizo y no hay nada que devolver.	La transferencia se anuló (borrado lógico).
400	Bad Request (solicitud incorrecta)	Los datos enviados no son válidos.	Faltan datos, el monto es cero o negativo, o las fechas están mal escritas.
401	Unauthorized (no autorizado)	No se sabe quién hace la solicitud: falta el token, es falso o ya venció.	Toda ruta /api/ sin un token válido, y un secreto incorrecto al pedir el token.
403	Forbidden (prohibido)	Se sabe quién es, pero no tiene permiso para esa acción.	La billetera intenta ver la bitácora o el auditor intenta transferir.
404	Not Found (no encontrado)	Lo que se pide no existe o no es visible para quien pregunta.	Cuenta fuera de la lista blanca o transferencia inexistente o anulada.
409	Conflict (conflicto)	La solicitud choca con el estado actual de los datos.	Código de operación repetido (idempotencia) o intento de corregir una transferencia que ya movió dinero.
422	Unprocessable Content (no procesable)	Los datos tienen el formato correcto, pero la operación no se puede hacer por una regla de negocio.	El núcleo rechazó la transferencia, por ejemplo por saldo insuficiente.
500	Internal Server Error (error interno)	Falló algo inesperado dentro de SITRA.	No está previsto en ningún flujo; el manejador de errores responde sin mostrar detalles internos.
503	Service Unavailable (servicio no disponible)	Un sistema del que se depende no está disponible por ahora; se puede reintentar.	El núcleo no respondió (la transferencia queda FALLIDA) o la base de datos canceló la operación por un bloqueo (deadlock).

Las Figuras 10 y 11 muestran extractos del código del controlador y del servicio de transferencias. En los extractos se ajustaron los saltos de línea para que quepan en la página y se resumieron con "…" los argumentos que se repiten; el código completo está en el repositorio.

Figura 10

Controlador de transferencias (capa de presentación, extracto)

```
/** CREATE: 201 CONFIRMADA · 422 RECHAZADA por el núcleo · 503 FALLIDA. */
@PostMapping
public ResponseEntity<TransferenciaResponse> registrar(
        @RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
        @RequestBody CrearTransferenciaRequest datos, HttpServletRequest peticion) {
    Transferencia t = servicio.registrar(clientId, datos, peticion.getRemoteAddr());
    return ResponseEntity.status(codigoSegunEstado(t, HttpStatus.CREATED))
                         .body(TransferenciaResponse.de(t));
}

/** UPDATE: corregir y reintentar una RECHAZADA o FALLIDA (ADR-004). */
@PutMapping("/{id}")
public ResponseEntity<TransferenciaResponse> actualizar(
        @RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId, @PathVariable Long id,
        @RequestBody ActualizarTransferenciaRequest datos, HttpServletRequest peticion) {
    Transferencia t = servicio.actualizar(clientId, id, datos, peticion.getRemoteAddr());
    return ResponseEntity.status(codigoSegunEstado(t, HttpStatus.OK))
                         .body(TransferenciaResponse.de(t));
}

/** DELETE lógico: 204 y la fila queda con activo = false y estado ANULADA. */
@DeleteMapping("/{id}")
public ResponseEntity<Void> anular(@RequestAttribute(FiltroJwt.ATRIBUTO_CLIENTE) String clientId,
                                   @PathVariable Long id, HttpServletRequest peticion) {
    servicio.anular(clientId, id, peticion.getRemoteAddr());
    return ResponseEntity.noContent().build();
}
```

Nota. Extracto de src/main/java/co/coop/sitra/controller/TransferenciaController.java. El controlador no contiene reglas de negocio: recibe la petición, llama al servicio y traduce el estado de la transferencia a un código HTTP.

Figura 11

Servicio de transferencias, operación de crear (capa de negocio, extracto)

```
@Transactional
public Transferencia registrar(String clientId, CrearTransferenciaRequest datos, String ip) {
    ClienteApi cliente = identificacion.exigirRol(clientId, Rol.BILLETERA);   // 403 si no lo es
    String cuenta = datos.cuentaOrigen();
    if (vacio(cuenta) || vacio(datos.cuentaDestino())
            || vacio(datos.referenciaExterna()) || datos.monto() == null) {
        throw rechazo(cliente, …, HttpStatus.BAD_REQUEST, "Faltan datos: …", ip);       // 400
    }
    if (datos.monto().compareTo(BigDecimal.ZERO) <= 0) {
        throw rechazo(cliente, …, HttpStatus.BAD_REQUEST, "El monto debe ser mayor que cero", ip);
    }
    if (transferencias.existsByReferenciaExterna(datos.referenciaExterna())) {
        throw rechazo(cliente, …, HttpStatus.CONFLICT, "El código de operación ya existe…", ip);
    }
    CuentaAutorizada origen = cuentas.findByNumeroCuentaAndActivoTrue(cuenta).orElse(null);
    if (origen == null) {
        throw rechazo(cliente, …, HttpStatus.NOT_FOUND, "La cuenta no está autorizada", ip);
    }
    Transferencia t = Transferencia.nueva(cliente, origen, datos.cuentaDestino(),
            datos.monto(), datos.referenciaExterna(), ip);
    ejecutarEnNucleo(t);                    // CONFIRMADA, RECHAZADA o FALLIDA
    try {
        return transferencias.save(t);      // INSERT: el trigger escribe la bitácora
    } catch (DataIntegrityViolationException e) {
        // Dos peticiones iguales al mismo tiempo: la llave UNIQUE frena la segunda
        throw rechazo(cliente, …, HttpStatus.CONFLICT, "Código de operación repetido", ip);
    }
}
```

Nota. Extracto de src/main/java/co/coop/sitra/service/TransferenciaService.java. El método rechazo(…) escribe el intento RECHAZADO en la bitácora en una transacción propia (REQUIRES_NEW), para que no se pierda aunque la operación principal se deshaga.

Cómo se ejecuta

Los pasos completos están en el README.md del repositorio. En resumen, en un equipo con Windows, XAMPP y Java 21:

1. En el panel de control de XAMPP, pulsar Start en MySQL (así se llama en XAMPP el servicio de MariaDB).
2. Crear la base de datos con datos de prueba. Por consola: `C:\xampp\mysql\bin\mysql.exe -u root --default-character-set=utf8mb4 < db\instalar_todo.sql`. Otra opción es abrir db/instalar_todo.sql en HeidiSQL y pulsar F9.

3. En la terminal de VS Code, con el perfil de Java 21, arrancar la API con `mvnw.cmd spring-boot:run`. El Maven Wrapper descarga Maven la primera vez, así que no hay que instalarlo. La API queda en http://localhost:8080.
4. En Postman, importar postman/coleccion-corte-vertical.json y ejecutar la colección completa con el Runner (19 peticiones en orden, 30 pruebas).
5. Correr las pruebas automáticas con `mvnw.cmd test` (pruebas unitarias, sin base de datos) y `mvnw.cmd test -Pintegracion` (todas, incluida la de integración con MariaDB).

5.4 Pruebas de la arquitectura

Las pruebas demuestran con evidencia repetible que la arquitectura cumple los escenarios de calidad de la Sección 1.3. Se entregan las dos modalidades que acepta la plantilla: una colección de Postman, que prueba la API desde afuera como lo haría la billetera, y pruebas automáticas JUnit, que prueban el código por dentro.

Colección de Postman

El archivo postman/coleccion-corte-vertical.json contiene 19 peticiones que se ejecutan en orden y 30 pruebas automáticas: cada petición comprueba por sí misma que la respuesta sea la esperada. La Tabla 15 las describe.

Tabla 15

Peticiones de la colección de Postman y lo que valida cada una

N.º	Petición	Resultado esperado	Qué valida	Pruebas
01	POST /auth/token con las credenciales de la billetera	200 y un token	La autenticación: el secreto se compara con su huella BCrypt y se entrega un token de tipo Bearer (portador).	2

N.º	Petición	Resultado esperado	Qué valida	Pruebas
02	POST /auth/token con un secreto incorrecto	401	AC-01: un secreto equivocado no recibe token.	1
03	POST /transferencias sin token	401	AC-01: sin token, la petición nunca llega al controlador.	1
04	POST /transferencias por 300.000	201, CONFIRMADA	Crear atravesando todas las capas; el núcleo devuelve un comprobante.	2
05	POST con el mismo código de operación	409	Idempotencia (AC-02): la operación no se duplica.	1
06	POST con monto de −50.000	400	Regla de negocio; el intento queda RECHAZADO en la bitácora.	1
07	POST desde la cuenta AH-9999	404	Lista blanca (mínimo privilegio): una cuenta no autorizada no existe para la billetera.	1
08	GET /cuentas/AH-1001/saldo	200 con el saldo	HU-02: el saldo se lee del núcleo.	2
09 y 10	GET de la lista y GET por id	200	Consultar: la transferencia creada aparece con la misma referencia.	4
11	PUT sobre la transferencia CONFIRMADA	409	ADR-004: lo que ya movió dinero no se corrige.	1
12	POST por 5.000.000	422, RECHAZADA	El núcleo rechaza por saldo insuficiente y queda el motivo.	2
13	PUT de la RECHAZADA con 100.000	200, CONFIRMADA	Actualizar: corregir y reintentar con el mismo código de operación.	2
14 y 15	DELETE de la de 300.000 y GET por id	204 y 404	Eliminar de forma lógica con reverso en el núcleo; ya no aparece entre las activas.	2
16 y 17	Token del auditor; bitácora con el token de la billetera	200 y 403	Autorización por rol: la billetera no puede ver la bitácora.	2
18	GET /auditoria?cuenta=AH-1001&desde=hoy&hasta=hoy	200 en menos de 5 s	HU-03 y métrica X = A / B de AC-01: la creada, los rechazos, la corrección y la anulación quedaron registrados, cada fila trae su huella y la consulta responde en menos de 5 segundos.	4
19	GET /auditoria/verificacion	200, íntegra = verdadero	Nadie alteró ni borró filas de la bitácora.	2

Nota. Total: 30 pruebas automáticas. La captura del Runner de Postman con el resultado se guarda en evidencias/entrega-07-pruebas-y-tradeoffs/Postman-Runner.png.

Pruebas automáticas con JUnit

Hay 21 pruebas JUnit en src/test/java: 17 unitarias, que prueban una pieza aislada imitando las demás con Mockito, y 4 de integración, que prueban la aplicación completa contra MariaDB. La Tabla 16 las describe.

Tabla 16

Pruebas JUnit del proyecto

Clase de prueba	Tipo	Qué comprueba	Cómo se corre
JwtServicioTest (6 pruebas)	Unitaria	Un token válido se acepta; un token alterado (por ejemplo, con el rol cambiado), firmado con otro secreto, vencido o mal formado se rechaza; con un secreto de menos de 32 caracteres la aplicación no arranca.	mvnw.cmd test
TransferenciaServiceTest (11 pruebas)	Unitaria con Mockito	Las respuestas 201, 400, 404, 409, 422 y 503 al crear; la transferencia FALLIDA sin efecto parcial (AC-02); el 409 del PUT sobre una CONFIRMADA y el reintento de una RECHAZADA; la anulación con reverso y el 503 sin anular. Corre sin núcleo y sin base de datos (AC-03).	mvnw.cmd test
CorteVerticalIntegracionTest (4 pruebas)	Integración (MockMvc y MariaDB)	El 401 sin token; el CRUD completo de punta a punta; el ciclo RECHAZADA, PUT y CONFIRMADA; el saldo, el 403 de la billetera registrado en la bitácora y la verificación íntegra de la cadena.	mvnw.cmd test -Pintegracion

Nota. MockMvc es una herramienta de Spring que envía peticiones HTTP a la aplicación sin necesidad de abrir el puerto 8080.

Archivos de evidencia

Las salidas de todas las pruebas se guardan como archivos de texto en evidencias/entrega-07-pruebas-y-tradeoffs/ para que cualquiera pueda revisarlas. La mayoría las genera de una sola vez el script scripts\generar_evidencias.cmd, que reinstala la base de datos y ejecuta cada prueba guardando su salida. La Tabla 17 las presenta.

Tabla 17

Archivos de evidencia de las pruebas

Archivo	**Qué contiene**	**Cómo se genera**
mvn-test.txt	Salida de las 17 pruebas unitarias JUnit, con el número de pruebas ejecutadas, fallidas y con error.	scripts\ generar_evidencias.cmd (paso 5), que ejecuta mvnw.cmd test.
mvn-test-integracion.txt	Salida de las 21 pruebas JUnit, incluida la de integración contra MariaDB.	scripts\ generar_evidencias.cmd (paso 6), que ejecuta mvnw.cmd test -Pintegracion.
resultado_verificacion_mariadb.txt	Respuestas de MariaDB a la demostración: bitácora escrita por el trigger, errores 1062, 4025 y 1644, rechazos registrados y cadena íntegra.	scripts\ generar_evidencias.cmd (paso 2), con db/pruebas/demo_sustentacion.sql.
resultado_minimo_privilegio.txt	Un usuario temporal con permisos mínimos intenta quitar un trigger y borrar la bitácora; los dos intentos fallan con el error 1142.	scripts\ generar_evidencias.cmd (paso 3); el usuario temporal se borra al terminar.
resultado_concurrencia.txt	Seis operaciones simultáneas sobre la bitácora en el equipo con XAMPP: cuántas se guardaron, cuántas se deshicieron por deadlock y si la cadena siguió íntegra.	scripts\ generar_evidencias.cmd (paso 4), con db/pruebas/concurrencia.cmd.
resultado_concurrencia_referencia.txt	La misma prueba ejecutada como referencia sobre MariaDB 10.11: 2 guardadas, 4 deshechas por deadlock y cadena íntegra.	db/pruebas/concurrencia.sh sobre una base recién instalada.
Postman-Runner.png	Captura del Runner de Postman con el resultado de las 19 peticiones y sus 30 pruebas.	Manualmente, con el Runner de Postman (paso 4 de “Cómo se ejecuta”).

El **criterio de aceptación** de esta sección es que cualquier persona, desde una terminal limpia y siguiendo solo el README.md, pueda instalar la base de datos, arrancar la API y ejecutar las pruebas (mvnw.cmd test, mvnw.cmd test -Pintegracion y el Runner de Postman) sin ayuda del autor.

Sección 6. Evaluación de la arquitectura y compromisos

Esta sección evalúa la arquitectura con los conceptos del método ATAM (Bass et al., 2021; Kazman et al., 2000): los **compromisos** (trade-offs), es decir, qué se sacrificó para ganar otra cosa, y los **puntos de sensibilidad**, las partes de la arquitectura que, si cambian o fallan, afectan directamente un atributo de calidad.

6.1 Análisis de compromisos

Toda arquitectura sacrifica algo. La Tabla 18 presenta el balance de SITRA: qué atributo se priorizó, cuál se sacrificó, qué evidencia hay del compromiso y por qué es aceptable en este proyecto.

Tabla 18

Compromisos (trade-offs) de la arquitectura

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	Por qué es aceptable en este proyecto
Seguridad y trazabilidad (AC-01)	Eficiencia de desempeño cuando hay muchas escrituras al mismo tiempo	Cada operación escribe una fila adicional en la bitácora, y el trigger debe leer la última huella antes de calcular la nueva. Cuando varias operaciones escriben a la vez, la base de datos las hace esperar y a veces cancela algunas por deadlock (ver la medición más abajo).	El volumen de operaciones de una cooperativa es bajo. Además, la operación cancelada se deshace completa y SITRA responde 503: la billetera la reintenta con el mismo código de operación sin riesgo de duplicarla (riesgo R-03).

Atributo priorizado	Atributo sacrificado	Evidencia del compromiso	Por qué es aceptable en este proyecto
Mantenibilidad (AC-03)	Rendimiento de las consultas	JPA genera las consultas automáticamente; al listar la bitácora, carga por separado el cliente y la transferencia de cada fila, lo que en programación se conoce como el problema de las consultas N+1: muchas consultas pequeñas en lugar de una sola grande.	La consulta del auditor siempre filtra por cuenta y fechas, y para eso existe el índice idx_auditoria_cuenta_fecha; con el volumen esperado se cumple la meta de 5 segundos (petición 18). Si el volumen creciera, se escribiría a mano esa única consulta.
Seguridad (AC-01)	Simplicidad y velocidad de desarrollo	El filtro JWT, la firma HMAC y BCrypt suman unas 240 líneas de código propio y un paso más en cada petición.	Se mueve dinero y se manejan datos personales protegidos por la Ley 1581 de 2012. Con esta solución se evita instalar y operar un servidor de identidad (ADR-005), y el código está cubierto por 6 pruebas unitarias.
Integridad de la evidencia (AC-01)	Flexibilidad del CRUD	El PUT solo acepta transferencias RECHAZADAS o FALLIDAS, y el DELETE exige el reverso en el núcleo (ADR-004). Una billetera no puede “editar” libremente una transferencia.	Una transferencia confirmada no puede maquillarse: la bitácora siempre coincide con lo que pasó en el núcleo, que es justamente lo que revisa el auditor.
Cero presupuesto (RA-02) y cero personal (RA-03)	Disponibilidad	Todo corre en un solo equipo existente, sin copias de respaldo en funcionamiento ni reparto de carga entre varios equipos (Sección 3.5). Si ese equipo se apaga, SITRA deja de atender.	La alta disponibilidad está fuera del alcance (1.1). Lo crítico es no perder la bitácora, y para eso basta un respaldo diario con mysqldump, la herramienta de copias que ya trae XAMPP.

Medición de la concurrencia

La **concurrencia** es la situación en la que varias operaciones ocurren al mismo tiempo. Para medir el primer compromiso de la Tabla 18 se versionó el script db/pruebas/concurrencia.sh, que lanza seis transacciones simultáneas que escriben en la bitácora y después cuenta cuántas quedaron guardadas y verifica la cadena de huellas. En la ejecución de referencia, sobre MariaDB 10.11 y una base recién instalada, se guardaron 2 de las 6

operaciones, las otras 4 se deshicieron por deadlock (error 1213) y la cadena de huellas quedó íntegra (archivo resultado_concurrencia_referencia.txt).

La versión de Windows del mismo script es db/pruebas/concurrencia.cmd, y su salida en el equipo con XAMPP se guarda en resultado_concurrencia.txt. El número de operaciones deshechas cambia en cada ejecución, porque depende del orden exacto en que lleguen las operaciones; lo que no cambia es el resultado que importa: ninguna operación queda a medias y la cadena de huellas nunca se rompe.

6.2 Puntos de sensibilidad y riesgos técnicos

La Tabla 19 identifica siete puntos de sensibilidad, el riesgo técnico de cada uno, su probabilidad e impacto, y la mitigación propuesta. La **probabilidad** indica qué tan posible es que el riesgo ocurra y el **impacto** qué tan grave sería si ocurriera.

Tabla 19

Puntos de sensibilidad y riesgos técnicos

N.º	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Probabilidad e impacto	Mitigación propuesta
R-01	Secreto de firma del token (SITRA_JWT_SECRET)	Seguridad	Si se filtra, cualquiera puede fabricar tokens válidos y hacerse pasar por la billetera o por el auditor.	Baja / crítico	Se guarda como variable de entorno fuera de Git (.env está en .gitignore); mínimo 32 caracteres; aleatorio si falta; tokens de 30 minutos; se cambia antes de pasar a producción.
R-02	Orden “primero el núcleo, después guardar” en TransferenciaService	Fiabilidad e integridad	El núcleo descuenta el dinero, pero SITRA falla al guardar: queda dinero movido sin transferencia registrada.	Baja / alto	Con el simulador, la misma transacción lo deshace. Con el núcleo real: enviarle el código de operación y conciliar a diario contra sus reportes; evaluar guardar la transferencia como PENDIENTE antes de llamar al núcleo.

N.º	Punto de sensibilidad	Atributo afectado	Riesgo técnico	Probabilidad e impacto	Mitigación propuesta
R-03	Trigger de la huella encadenada, que lee la última fila de la bitácora	Fiabilidad y eficiencia	Con escrituras simultáneas, la base de datos bloquea filas y aparecen deadlocks (medido: 4 de 6 en la ejecución de referencia).	Media si crece el volumen / medio	ManejadorErrores responde 503, que se puede reintentar; la idempotencia evita duplicados. Si el volumen crece: reintento automático o una sola cola de escritura.
R-04	Credenciales de la base de datos (root sin contraseña en XAMPP)	Seguridad	Alguien con acceso total a la base desactiva los triggers y altera la bitácora.	Media / crítico	Usuario sitra_app sin permisos de borrar ni de manejar triggers (probado: error 1142, en resultado_minimo_privilegio.txt); la verificación de huellas detecta cualquier cambio (probado: ALTERADO y ROTA).
R-05	Comunicación por HTTP en el puerto 8080	Seguridad (confidencialidad)	El token y los datos viajan sin cifrar y alguien en la misma red podría leerlos.	Alta en el laboratorio / alto	Activar HTTPS en Spring Boot con un certificado gratuito antes de producción (deuda técnica del ADR-005).
R-06	Protocolo del núcleo sin confirmar (NucleoLegadoConector)	Adecuación funcional	No lograr conectar el núcleo real dentro de los 6 meses del plazo regulatorio.	Media / crítico	El conector está aislado (ADR-001): solo cambia una clase. Confirmar el protocolo es la primera tarea con el cliente, y con él se implementa la espera máxima de 5 segundos (AC-02).
R-07	MariaDB 10.4 de XAMPP, que ya no recibe actualizaciones	Seguridad	Quedan vulnerabilidades conocidas sin corregir.	Media / medio	Actualizar a MariaDB 10.11, versión con soporte extendido; los scripts ya se ejecutaron en 10.11 sin cambios.

6.3 Conclusión arquitectónica

La plantilla pide responder dos preguntas: si la arquitectura satisface los escenarios de calidad y qué evolucionaría primero si el proyecto pasara a producción. Las respuestas son las siguientes:

- **Sí satisface los tres escenarios de la Sección 1.3, con evidencia del corte vertical.** En AC-01, sin token se responde 401 antes de llegar a los controladores (peticiones 02 y 03 y pruebas JUnit); el 100 % de las operaciones, incluidas las rechazadas, quedan en la bitácora porque la base de datos la escribe con triggers y el servicio registra los rechazos en una transacción propia (petición 18), y nadie puede modificarla sin que la verificación de huellas lo detecte (petición 19 y paso 5 de la demostración). En AC-02, si el núcleo no responde, la transferencia queda FALLIDA con 503 y sin efecto parcial, se puede reintentar con el mismo código (PUT) y un código repetido nunca duplica la operación (409). En AC-03, el núcleo se cambia con una variable (SITRA_NUCLEO_MODO) y las pruebas unitarias corren sin núcleo y sin base de datos.
- **Lo primero que evolucionaría es el conector real del núcleo.** En cuanto el cliente confirme cómo expone sus datos el núcleo, se implementa la clase NucleoLegadoConector con la espera máxima de 5 segundos, junto con la conciliación diaria contra los reportes del núcleo (riesgos R-02 y R-06). Después vendría el endurecimiento del despliegue: HTTPS, el usuario sitra_app de mínimo privilegio, un secreto JWT definido y rotado, MariaDB 10.11 y respaldos automáticos (riesgos R-01, R-04, R-05 y R-07). Ninguno de esos cambios toca las capas de presentación ni de negocio, y esa es la prueba de que la estructura elegida funciona.

Conclusiones

Del trabajo realizado se derivan cinco conclusiones:

- **Se puede cumplir la exigencia del regulador sin modificar el núcleo legado.** El conector intercambiable (ADR-001) permite construir y probar toda la integración contra

un simulador, y deja la conexión real reducida a una sola clase que se implementará cuando el cliente confirme cómo expone sus datos el núcleo.

- **La trazabilidad es más confiable cuando la garantiza la base de datos y no solo el programa.** Los triggers, la prohibición de modificar o borrar la bitácora y las huellas encadenadas (ADR-003) hacen que ninguna operación, ni siquiera una rechazada o una hecha directamente en la base, quede sin evidencia, y que cualquier alteración se detecte.
- **Las restricciones de cero presupuesto y cero personal técnico definieron la tecnología tanto como los atributos de calidad.** Por eso se eligieron herramientas gratuitas que ya existen en la cooperativa (MariaDB de XAMPP, Java), una estructura estándar que cualquier programador reconoce y un despliegue de un solo archivo, sin plataformas nuevas que instalar ni operar.
- **Toda decisión tuvo un costo y quedó documentada.** Se sacrificó desempeño en escrituras simultáneas, flexibilidad del CRUD y disponibilidad a cambio de seguridad, trazabilidad y mantenibilidad; los cinco compromisos y los siete riesgos quedaron medidos o justificados, con su mitigación, en la Sección 6.
- **Queda una decisión que no es técnica.** La pregunta de quién operará y sostendrá la integración, y con qué recursos, le corresponde a la junta directiva de la cooperativa (ver 1.4); la arquitectura reduce al mínimo ese esfuerzo, pero no puede eliminarlo.

Referencias

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Brown, S. (s.f.). *The C4 model for visualising software architecture*. https://c4model.com

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Corporación Universitaria Remington. (2026). *Plantilla del Documento de Arquitectura de Software (DAS)* [Material del curso Arquitectura de Software].

Fielding, R., Nottingham, M. y Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). https://www.iso.org/standard/35733.html

International Organization for Standardization. (2016). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality* (ISO/IEC 25023:2016). https://www.iso.org/standard/35747.html

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Kazman, R., Klein, M. y Clements, P. (2000). *ATAM: Method for architecture evaluation* (Informe técnico CMU/SEI-2000-TR-004). Software Engineering Institute, Carnegie Mellon University.

Krawczyk, H., Bellare, M. y Canetti, R. (1997). *HMAC: Keyed-hashing for message authentication* (RFC 2104). Internet Engineering Task Force. https://doi.org/10.17487/RFC2104

MariaDB plc. (s.f.). *MariaDB Server documentation*. https://mariadb.com/docs/server

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Provos, N. y Mazières, D. (1999). A future-adaptable password scheme. En *Proceedings of the 1999 USENIX Annual Technical Conference* (pp. 81–91). USENIX Association.

Spring. (2024). *Spring Boot reference documentation* (versión 3.3). https://docs.spring.io/spring-boot/3.3/

Starke, G. y Hruschka, P. (s.f.). *arc42: Plantilla para la documentación de arquitecturas de software*. https://arc42.org

Anexo A

Lista de chequeo de entrega

La Tabla A1 verifica los doce entregables de la lista de chequeo final.

Tabla A1

Lista de chequeo de entrega

N.º	Entregable	¿Incluido?	Dónde se encuentra
1	DAS completo (Secciones 1 a 6) en PDF dentro de docs/	Sí	docs/DAS.pdf y docs/DAS.docx (versión 1.1); copia en evidencias/entrega-08-entrega-final/.
2	Tres escenarios de calidad ISO/IEC 25010 con métricas	Sí	Sección 1.3, Tabla 4 (AC-01, AC-02 y AC-03).
3	Cinco vistas: contexto, contenedores, clases, secuencia y despliegue	Sí	Sección 3 (Figuras 1 a 5) y docs/diagramas/.
4	Mínimo un ADR en docs/adr/ con la plantilla oficial	Sí (cinco)	docs/adr/ADR-001 a ADR-005 y Sección 4.
5	Repositorio Git con al menos 10 commits atómicos y .gitignore correcto	Sí	Historial con Conventional Commits (Sección 5.1); el .gitignore excluye .env y target/.
6	Script SQL con PK, FK, CHECK, UNIQUE y borrado lógico	Sí	db/migrations/V1 a V4 (Sección 5.2, Tabla 12).
7	CRUD de punta a punta ejecutable (modelo, controlador, rutas y API)	Sí	Sección 5.3; src/main/java.
8	Colección de Postman o pruebas de integración pasando	Sí (ambas)	postman/ (30 pruebas) y src/test/java (21 pruebas); salidas en evidencias/entrega-07-pruebas-y-tradeoffs/ (Sección 5.4).
9	README.md que permite instalar y ejecutar sin ayuda del autor	Sí	README.md, sección “Instalar y ejecutar”, con Maven Wrapper y creación de la base por consola.
10	Análisis de compromisos y al menos tres puntos de sensibilidad	Sí (cinco y siete)	Sección 6.1 (Tabla 18) y Sección 6.2 (Tabla 19).
11	Toda la documentación versionada en docs/	Sí	docs/DAS.docx, docs/DAS.pdf, docs/adr/ y docs/diagramas/ (fuentes .puml e imágenes).
12	Enlace del repositorio en la portada; repositorio público	Sí	Portada de este documento: https://github.com/JorgeMario-V/sitra (público).

Anexo B

Correcciones de la versión 1.1

La versión 1.1 corrige los doce hallazgos de la revisión de trazabilidad hecha por la docente. Las correcciones se hicieron en la rama feature/correcciones-trazabilidad, con un commit por hallazgo y sin reescribir el historial. La Tabla B1 resume cada hallazgo y dónde se ve su corrección.

Tabla B1

Hallazgos de la revisión y su corrección

Hallazgo	Qué se pidió	Qué se hizo	Dónde se ve
H-01	La lista de chequeo debe tener 12 ítems.	Se agregaron el ítem 11 (documentación versionada en docs/) y el 12 (enlace del repositorio en la portada y repositorio público).	Anexo A, Tabla A1.
H-02	Explicar cómo crear la base de datos por consola.	El README ofrece la opción por consola con mysql.exe y utf8mb4, además de la opción con HeidiSQL.	README.md, paso 2; Sección 5.3.
H-03	Que el proyecto se pueda ejecutar sin instalar Maven.	Se agregó el Maven Wrapper 3.3.2 (mvnw, mvnw.cmd y .mvn/); los comandos pasan a ser mvnw.cmd spring-boot:run y mvnw.cmd test.	Raíz del repositorio; README.md; Secciones 5.3 y 5.4.
H-04	Guardar las salidas de las pruebas JUnit.	Se creó scripts\ generar_evidencias.cmd, que guarda mvn-test.txt y mvn-test-integracion.txt.	evidencias/entrega-07-pruebas-y-tradeoffs/; Tabla 17.
H-05	Evidencia de verificación en MariaDB; marcar como histórica la de PostgreSQL.	El script genera resultado_verificacion_mariadb.txt y resultado_minimo_privilegio.txt; el archivo de la entrega 4 lleva una nota de archivo histórico.	evidencias/entrega-07 y entrega-04; Sección 5.2.
H-06	Medir el tiempo de respuesta de la consulta del auditor.	La petición 18 de Postman comprueba que responde en menos de 5 segundos; la colección pasa a tener 30 pruebas y se toma una nueva captura del Runner.	postman/; Tablas 4 y 15.

Hallazgo	Qué se pidió	Qué se hizo	Dónde se ve
H-07	Versionar el script de concurrencia y guardar su salida.	Se agregaron db/pruebas/concurrencia.sh y concurrencia.cmd con su salida de referencia y la del equipo con XAMPP.	db/pruebas/; Sección 6.1.
H-08	No reescribir el historial y aclarar desde qué clase se usan ramas.	Sin rebase ni push forzado; la Sección 5.1 aclara que las ramas feature/ se usan desde la Clase 5 y que las Clases 1 a 4 se hicieron en main.	Sección 5.1.
H-09	Agregar la fecha de subida a GitHub y corregir fechas.	La tabla de versiones tiene la columna “Subida a GitHub”; el README de evidencias dice 16/10 y las Clases 7 y 8 figuran como adelantadas (06/11).	Tabla 1; README.md y evidencias/README.md.
H-10	Aclarar la espera de 5 segundos de AC-02 (recomendado).	Se aclaró que es una meta pendiente del conector real (deuda técnica del ADR-001) y que hoy se verifica el resultado, no el tiempo.	Sección 1.3, Tabla 4.
H-11	Transcribir la pregunta al cliente (recomendado).	Se transcribió la pregunta del punto 5 de la ficha de drivers y el supuesto oculto que revela.	Sección 1.4.
H-12	Eliminar la página en blanco entre 3.2 y 3.3 (recomendado).	El documento ya no usa saltos de página manuales, salvo los que exige APA (portada, resumen, contenido, inicio del texto, referencias y anexos).	Sección 3.

Nota. Además, la versión 1.1 aplica el formato APA 7: portada con la docente y el repositorio, resumen y palabras clave, glosario de términos, títulos en negrita, tablas y figuras numeradas con su título y nota, explicación de los términos técnicos, conclusiones separadas con guiones y referencias.